

Foundations of Multi-Designated Verifier Signature Comprehensive Formalization and New Constructions in Subset Simulation*

Keitaro Hashimoto¹, Kyosuke Yamashita^{2,1}, Keisuke Hara^{1,3}

¹National Institute of Advanced Industrial Science and Technology (AIST), Tokyo, Japan

{keitaro.hashimoto, hara-keisuke}@aist.go.jp

²The University of Osaka, Osaka, Japan

yamashita@ist.osaka-u.ac.jp

³Yokohama National University, Kanagawa, Japan

June 17, 2025

Abstract

A multi-designated verifier signature (MDVS) is a digital signature that empowers a signer to designate specific verifiers capable of verifying signatures. Notably, designated verifiers are allowed to not only verify signatures but also simulate “fake” signatures indistinguishable from real ones produced by the original signer. Since this property is useful for realizing off-the-record (i.e., deniable) communication in group settings, MDVS is attracting attention in secure messaging. Recently, Damgård et al. (TCC’20) and Chakraborty et al. (EUROCRYPT’23) have introduced new MDVS schemes, allowing a subset of designated verifiers to simulate signatures in contrast to the conventional one, which requires all designated verifiers for signature simulation. They also define a stronger notion of security for them. This work delves into this new MDVS and offers a comprehensive formalization. We identify all possible security levels of MDVS schemes in subset simulations and prove that some of them are not feasible. Furthermore, we demonstrate that MDVS schemes meeting the security notion defined by Chakraborty et al. imply IND-CCA secure public-key encryption schemes. Beyond formalization, we present new constructions of MDVS schemes in subset simulation. Notably, we introduce a new construction of strongly secure MDVS schemes based on ring signatures and public-key encryption, accompanied by a generic conversion for achieving consistency through non-interactive zero-knowledge arguments. Finally, we evaluate the efficiency of our MDVS schemes in classical and post-quantum settings, showing their practicality.

1 Introduction

1.1 Background

A multi-designated verifier signature (MDVS) [LV04] is a digital signature allowing a signer to designate verifiers who can verify the signer’s signatures. The designated verifiers use their secret verification key for signature verification. The fundamental security prerequisites for MDVS encompass unforgeability, consistency, and off-the-record (OTR). Notably, OTR is a distinctive attribute of MDVS, allowing the designated verifiers to simulate “fake” signatures indistinguishable from the signer’s original signatures.

*A preliminary version of this paper appears in the proceedings of the 38th IEEE Computer Security Foundations Symposium (IEEE CSF 2025). This is the full version.

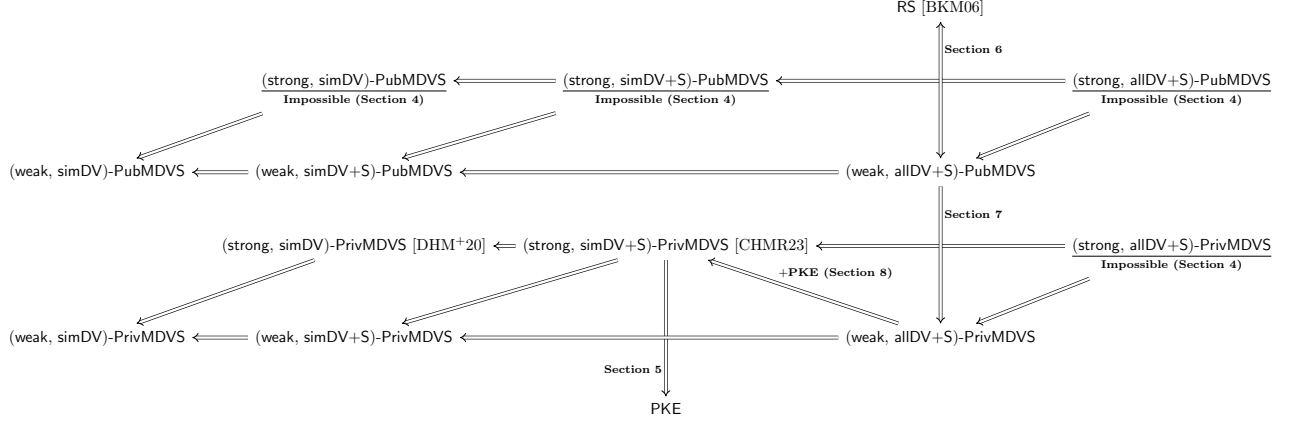


Figure 1: Relationship among 12 types of subset simulatable multi-designated verifier signature MDVS, ring signature RS, and public-key encryption PKE. For the prefix (xxx, yyy), xxx stands for the level of unforgeability (adversary cannot or can generate simulated signatures (*weak* or *strong*)), and yyy stands for the level of off-the-record (*simDV*, *simDV+S*, or *allDV+S*). “ $A \Rightarrow B$ ” means that the primitive B can be constructed from the primitive A (with some additional primitives). The arrows without labels have trivial implications. Note that this figure does not care about consistency.

There are many real-world applications of MDVS. A promising one is secure group messaging. As an essential property of secure group messaging, deniability is considered in the literature [MPR22, DHM⁺20, CHMR23]. By authenticating the exchanged messages with MDVS, the OTR property ensures senders to claim the signature for the message is a fake one simulated by the other members, and thus, no one can determine whether the sender sent the message. As a result, senders can enjoy off-the-record communication on the Internet. Also, MDVS is useful to realize content moderation [TGL⁺19] in secure group messaging. In a secure messaging system, malicious users might exploit the security of it to send illegal or harmful content. Content moderation is a mechanism that allows a moderator to identify the senders of such content and exclude them from the system. MDVS can realize this mechanism as follows. When a sender sends a message, the sender authenticates it with an MDVS signature by designating the intended receivers and the moderator as verifiers. If the receivers notice the contents are illegal/harmful, they report the fact to the moderator. Since the moderator is designated, it can verify that the sender sends the content. Here, unforgeability ensures that the sender generates the signature, not forged by the reported receiver. Moreover, OTR property allows the sender to deny that it sent such content to anyone other than the receivers and the moderator. Finally, MDVS is useful to realize watermarking for large language models (LLMs) [HZM⁺24]. By using MDVS, the authors construct multi-designated detector watermarking scheme that allows model providers to generate watermarked outputs from LLMs so that only specific designated detectors can distinguish between texts generated by LLMs and those written by humans.

MDVS garnered significant attention in the 2000s [LV04, Cho08, LSMP07, LV07, SKS06], but they approached MDVS informally summarized in Table 1. Afterwards, Zhang et al. [ZAYS12] provided a formal definition of MDVS. These works considered unforgeability against an adversary unable to execute the simulator and OTR against an adversary that could know all secret keys. These MDVS schemes focused on *all simulation*, where signature simulation requires the secret keys of all designated verifiers. Damgård et al. [DHM⁺20] introduced a new MDVS scheme allowing *subset simulation*, wherein any subset of designated verifiers, or even just one of them, can simulate fake signatures. Since any designated verifier can create a fake signature, it is difficult to guess whether a given signature is real or simulated. Damgård et al. [DHM⁺20] also defined the security notion for subset simulatable MDVS (cf. Table 1). They termed the conventional unforgeability in [ZAYS12] as *weak unforgeability* and introduced *strong unforgeability*, where an adversary is allowed to run the simulator. In other words, a designated verifier can distinguish real signatures from simulated ones generated by other verifiers. Concerning OTR, Damgård et al. defined a restricted version

Table 1: Summary of prior work. “—” indicates that the work did not formally define the property. The column ‘OTR’ stands for secret keys the adversary knows.

Reference	Simulation	Unforgeability	Off-the-record	Consistency
[LV04]	all	weak	—	—
[SKS06, LV07], [LSMP07, Cho08]	all	—	—	—
[ZAYS12]	all	weak ¹	allDV+S	—
[DHM ⁺ 20]	subset	strong	simDV	✓
[CHMR23]	subset	strong	simDV+S	✓

where the adversary only knows the secret keys of the verifiers used for simulation. In other words, their OTR security disallowed corruption of the signer and the other verifiers. They also defined *consistency*, guaranteeing that an adversary who knows the secret signing key cannot forge a signature, causing an inconsistent view for designated verifiers. Subsequent work by Chakraborty et al. [CHMR23] introduced an enhanced OTR, ensuring that real and simulated signatures remain indistinguishable even if the signer is corrupted along with the verifiers involved in the simulation.

As observed from Table 1, different security concepts are outlined in the context of MDVS, yet they have only been deliberated in isolation. This results in an ambiguous overall understanding of MDVS. Especially, it remains many open questions in subset simulation:

- Can we construct weakly unforgeable schemes, OTR secure schemes even when all verifier’s secret keys are leaked, and schemes without consistency?
- Can we realize any combinations of unforgeability and OTR property?
- What is the relationship between subset simulatable MDVS and other primitives?

1.2 Our Contributions

In this work, we delve into the subset simulatable MDVS² [DHM⁺20, CHMR23] and answer the above questions, especially showing the relationships among the security notions and between MDVS and other cryptographic primitives.

Redefining subset simulatable MDVS. We first provide a comprehensive formal definition of MDVS. We consider all existing security notions proposed in the recent works [DHM⁺20, CHMR23] and the conventional work [ZAYS12]. Now, we have two flavors of unforgeability (*weak* and *strong*) and three flavors of off-the-record (*simDV*, *simDV+S*, and *allDV+S*), respectively. The notation for OTR stands for what types of secret keys an adversary knows: designated verifiers that simulate signatures (*simDV*), designated verifiers that simulate signatures, and a signer (*simDV+S*), and all designated verifiers and a signer (*allDV+S*). In addition to them, we consider publicly verifiable MDVS, where anyone can verify signatures without secret keys³. We call privately (resp., publicly) verifiable MDVS PrivMDVS (resp., PubMDVS). As a result, we identify 12 types of formalization⁴ in MDVS as depicted in Figure 1. Significantly, this work is the first one that formally defines *weak* and *allDV+S* security on subset simulatable MDVS. It is worth noting that (*strong*, *simDV*)-PrivMDVS and (*strong*, *simDV+S*)-PrivMDVS are identical to the definition in [DHM⁺20] and [CHMR23], respectively.

¹Whereas Damgård et al. [DHM⁺20] mentioned that [ZAYS12] has strong unforgeability, we claim that it is weak unforgeable since [ZAYS12] only considered an adversary who cannot run the simulator to forge a signature.

²This work only focuses on subset simulatable MDVS, so MDVS indicates subset simulatable MDVS throughout this paper.

³The motivation of introducing public verification is to examine the relationship between publicly verifiable MDVS and ring signature as the known result that publicly verifiable DVS and 2-user ring signature are equivalent [HKP22, BFG⁺22].

⁴For each type, the presence or absence of consistency can be considered.

Table 2: Comparison of the existing subset simulatable PrivMDVS schemes and our schemes. Note that AVPKE stands for authenticated and verifiable PKE.

Scheme	Unf.	OTR	Con.	Tools
[DHM ⁺ 20]	strong	simDV	✓	PRG, Com, NIKE, NIZK-PoK
[DHM ⁺ 20]	strong	simDV	✓	AVPKE, NIZK
[DHM ⁺ 20]	strong	simDV	✓	Functional encryption, NIZK
[CHMR23]	strong	simDV+S	✓	OWF, PKE, NIZK
Ours 1	weak	allDV+S	✓	RS
Ours 2	strong	simDV+S	✗	RS, PKE, OTS
Ours 3	strong	simDV+S	✓	RS, PKE, OTS, NIZK

Drawing a whole picture of MDVS. We reveal impossibilities, relationships among security notions, and relationships with other cryptographic primitives for subset simulatable MDVS. The details are as follows.

1. It is impossible to realize (strong, allDV+S)-PrivMDVS and (strong, *)-PubMDVS. (Cf. Section 4)
2. (strong, simDV+S)-PrivMDVS (without consistency) implies *IND-CCA* secure public-key encryption (PKE). (Cf. Section 5)
3. (weak, allDV+S)-PubMDVS is equivalent to ring signature (RS).⁵ (Cf. Section 6)
4. (weak, yyy)-PrivMDVS can be constructed from (weak, yyy)-PubMDVS where $yyy \in \{\text{simDV}, \text{simDV}+S, \text{allDV}+S\}$. (Cf. Section 7)
5. (strong, simDV+S)-PrivMDVS (without consistency) can be constructed from (weak, allDV+S)-PrivMDVS plus *IND-CCA* secure PKE and strong one-time signature (OTS). (Cf. Section 8)
6. strong-Unf or allDV+S-OTR secure PrivMDVS schemes can be added consistency by using non-interactive zero-knowledge arguments (NIZK). (Cf. Section 9)

From these results, we obtain the following conclusions. First, as one of the key limitations of MDVS, we cannot obtain (strong, allDV+S)-PrivMDVS and (strong, allDV+S)-PrivMDVS by any means. We believe that this result gives a clear research direction on MDVS in the future. Second, PubMDVS is *fully equivalent* to RS. In other words, PubMDVS is just a paraphrase of RS with no essential difference. Therefore, introducing and using PubMDVS is redundant. Third, we get a new paradigm for constructing (strong, simDV+S)-PrivMDVS (without consistency) based on RS, *IND-CCA* secure PKE, and strong OTS. This result also reveals that NIZK is unnecessary if consistency is not required.

Table 2 summarizes the existing PrivMDVS schemes and our three schemes derived from the above results (3), (4), (5), and (6). Compared to the previous works, all of our proposed schemes have their own remarkable properties. Our first scheme (Ours 1) can be simply constructed based on RS. While this scheme satisfies only weak unforgeability, it can offer (the strongest) allDV+S OTR property. Since we have a lot of work on constructing efficient RS schemes to date (e.g., [LPQ18, YEL⁺21]), we can obtain various efficient (weak, allDV+S)-PrivMDVS schemes. Our second scheme (Ours 2) provides a generic method to obtain a (strong, simDV+S)-PrivMDVS scheme *without consistency*. All the previous subset simulatable PrivMDVS schemes were designed to satisfy consistency as default. Compared to them, due to the lack of consistency, our second scheme has a simpler structure (and nice efficiency), especially because it does not require any NIZK proof system. Finally, our third scheme (Ours 3) offers a new methodology to obtain a (strong, simDV+S)-PrivMDVS scheme with consistency (that is, a most desirable MDVS scheme). Although previous methodologies to offer consistency require too heavy building blocks or combine a basic primitive in a complex manner, our third scheme is derived from a more simple and efficient way to upgrade a (strong, simDV+S)-PrivMDVS scheme without consistency (such as our second scheme) into a full-fledged one.

⁵Our result can be seen as a generalization of the previous result [HKKP22, BFG⁺22].

1.3 Organization of This Paper

First, we introduce notations and basic cryptographic primitives in Section 2. In Section 3, we provide a comprehensive definition of subset simulatable MDVS. We explain syntax and security definitions for subset simulatable MDVS in detail. Then, in Section 4, we proved some properties are incompatible, showing the impossibility of some types of MDVS schemes. Sections 5 and 6 demonstrates the relationship between MDVS and other cryptographic primitives. Sections 7 to 9 demonstrates the relationship among subset simulatable MDVS. These sections explain how to convert an MDVS scheme into another MDVS scheme with different properties. Finally, Section 10 evaluates the communication costs of our MDVS schemes. Conclusion and open problems are provided in Section 11.

2 Preliminaries

2.1 Notation

$\kappa \in \mathbb{N}$ denotes a security parameter. $\text{poly}(\cdot)$ and $\text{negl}(\cdot)$ are any polynomial function and negligible function, respectively. For $n \in \mathbb{N}$, we define $[n] := \{1, 2, \dots, n-1, n\}$. For a set S and an element s , we use $S \leftarrow s$ as the syntax sugar of $S \leftarrow S \cup \{s\}$. For a list of tuples $L = \{(x, y)\}$, we write $y \leftarrow L[x]$ to take an element y corresponding to x . PPT stands for probabilistic polynomial time. $y \leftarrow \mathcal{A}(x; r)$ denotes that a PPT algorithm $\mathcal{A}$ outputs y on input x and randomness r . We simply denote $y \leftarrow \mathcal{A}(x)$ when $\mathcal{A}$ uses uniform randomness.

2.2 Ring Signature

Definition 1 (Ring Signature). *A ring signature scheme RS consists of the following four algorithms:*

$\text{Setup}(1^\kappa) \rightarrow \text{pp}$: *The probabilistic setup algorithm takes a security parameter 1^κ as input and outputs a public parameter pp. Note that pp defines a message space $\mathcal{M}$.*

$\text{KGen}(\text{pp}) \rightarrow (\text{vk}, \text{sk})$: *The probabilistic key generation algorithm takes a public parameter pp as input and outputs a pair of verification and signing keys (vk, sk) .*

$\text{Sign}(\text{pp}, \text{sk}, \text{m}, \text{R}) \rightarrow \sigma$: *The probabilistic signing algorithm takes a public parameter pp, a signing key sk, a message m, and a list of verification keys, i.e., a ring, $\text{R} := \{\text{vk}_i\}_{i \in [N]}$ as input and outputs a signature σ (for some $N = \text{poly}(\kappa)$).*

$\text{Verify}(\text{pp}, \text{R}, \text{m}, \sigma) \rightarrow 1/0$: *The deterministic verification algorithm takes a public parameter pp, a ring R, a message m, and a signature σ as input and outputs 1 or 0.*

We require a ring signature scheme RS to satisfy correctness, unforgeability, and anonymity, defined as follows.

Definition 2 (Correctness). *We say that an RS scheme RS is correct if for any $N = \text{poly}(\kappa)$, all $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, all keys $(\text{vk}_i, \text{sk}_i) \leftarrow \text{KGen}(\text{pp})$ ($i \in [N]$), and any message $\text{m} \in \mathcal{M}$, it holds that $\text{Verify}(\text{pp}, \text{R}, \text{m}, \sigma) = 1$ with overwhelming probability, where $\text{R} := \{\text{vk}_j\}_{j \in [N]}$ and $\sigma \leftarrow \text{Sign}(\text{pp}, \text{sk}_i, \text{R}, \text{m})$.*

Definition 3 (Unforgeability). *Let RS be an RS scheme. We define the game $\text{Game}_{\text{RS}, \mathcal{A}}^{\text{Unf}}$ in Figure 2 and the advantage of $\mathcal{A}$ as*

$$\text{Adv}_{\text{RS}, \mathcal{A}}^{\text{Unf}}(\kappa) := \Pr \left[\text{Game}_{\text{RS}, \mathcal{A}}^{\text{Unf}}(\kappa) = 1 \right].$$

We say a ring signature scheme RS is unforgeable if $\text{Adv}_{\text{RS}, \mathcal{A}}^{\text{Unf}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

$\text{Game}_{\text{RS}, \mathcal{A}}^{\text{Unf}}(\kappa)$	$\text{Game}_{\text{RS}, \mathcal{A}}^{\text{Anon}}(\kappa)$	$\text{O}_{\text{VK}}()$
1 : $L_{\text{VK}}, L_{\text{SK}}, L_{\text{Sig}} \leftarrow \emptyset$	1 : $L_{\text{VK}}, L_{\text{SK}} \leftarrow \emptyset$	1 : $(\text{vk}, \text{sk}) \leftarrow \text{KGen}(\text{pp})$
2 : $\text{pp} \leftarrow \text{Setup}(1^\kappa)$	2 : $b \leftarrow \mathbb{S} \{0, 1\}$	2 : $L_{\text{VK}} \leftarrow (\text{vk}, \text{sk})$
3 : $\mathcal{O} := \{\text{O}_{\text{VK}}, \text{O}_{\text{SK}}, \text{O}_{\text{Sig}}\}$	3 : $\text{pp} \leftarrow \text{Setup}(1^\kappa)$	3 : return vk
4 : $(\text{R}^*, \text{m}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp})$	4 : $(\text{vk}_0^*, \text{vk}_1^*, \text{R}^*, \text{m}^*) \leftarrow \mathcal{A}^{\text{O}_{\text{VK}}, \text{O}_{\text{SK}}}(\text{pp})$	$\text{O}_{\text{SK}}(\text{vk})$
5 : if $\text{Verify}(\text{pp}, \text{R}^*, \text{m}^*, \sigma^*) = 1$ $\wedge \forall \text{vk} \in \text{R}^* :$ $(\text{vk}, \text{sk}) \in L_{\text{VK}} \wedge \text{vk} \notin L_{\text{SK}}$ $\wedge (\text{vk}, \text{m}^*, \text{R}^*) \notin L_{\text{Sig}}$ then	5 : if $\{\text{vk}_0^*, \text{vk}_1^*\} \subseteq \text{R}^*$ $\wedge \forall \text{vk} \in \text{R}^* : (\text{vk}, \text{sk}) \in L_{\text{VK}}$	1 : if $(\text{vk}, *) \notin L_{\text{VK}}$ then
6 : return 1	6 : continue	2 : return $\perp$
7 : else	7 : else	3 : $\text{sk} \leftarrow L_{\text{VK}}[\text{vk}]; L_{\text{SK}} \leftarrow \text{vk}$
8 : return 0	8 : return b	4 : return sk
	9 : foreach $i \in \{0, 1\}$ do	$\text{O}_{\text{Sig}}(\text{vk}, \text{m}, \text{R})$
	10 : $\text{sk}_i \leftarrow L_{\text{VK}}[\text{vk}_i]$	1 : if $\text{vk} \notin \text{R}$ then return $\perp$
	11 : $\sigma_i^* \leftarrow \text{Sign}(\text{pp}, \text{sk}_i^*, \text{m}^*, \text{R}^*)$	2 : if $\exists \text{vk}' \in \text{R} : (\text{vk}', \text{sk}') \notin L_{\text{VK}}$ then
	12 : $b' \leftarrow \mathcal{A}^{\text{O}_{\text{VK}}, \text{O}_{\text{SK}}}(\sigma_b^*)$	3 : return $\perp$
	13 : if $b = b'$ then	4 : $\text{sk} \leftarrow L_{\text{VK}}[\text{vk}]$
	14 : return 1	5 : $\sigma \leftarrow \text{Sign}(\text{pp}, \text{sk}, \text{m}, \text{R})$
	15 : else	6 : $L_{\text{Sig}} \leftarrow (\text{vk}, \text{m}, \text{R})$
	16 : return 0	7 : return σ

Figure 2: Security games for ring signatures.

Definition 4 (Anonymity). Let RS be an RS scheme. We define the game $\text{Game}_{\text{RS}, \mathcal{A}}^{\text{Anon}}$ in Figure 2 and the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{RS}, \mathcal{A}}^{\text{Anon}}(\kappa) := \left| \Pr \left[\text{Game}_{\text{RS}, \mathcal{A}}^{\text{Anon}}(\kappa) = 1 \right] - \frac{1}{2} \right|.$$

We say a ring signature scheme RS is anonymous if $\text{Adv}_{\text{RS}, \mathcal{A}}^{\text{Anon}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

Remark 1 (On signing oracle in the unforgeability game). The above definitions consider a weaker signing oracle that only accepts honest rings (cf. line 2 of O_{Sig} in Figure 2). Such a signing oracle is sufficient for considering relationships between RS and MDVS. See also Remark 2.

2.3 Public-Key Encryption

Definition 5 (Public-Key Encryption). A public-key encryption scheme PKE consists of the following four algorithms:

$\text{Setup}(1^\kappa) \rightarrow \text{pp}$: The probabilistic setup algorithm takes a security parameter 1^κ as input and outputs a public parameter pp. Note that pp defines a message space $\mathcal{M}$.

$\text{KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$: The probabilistic key generation algorithm takes a public parameter pp as input and outputs a pair of encryption and decryption keys (pk, sk).

$\text{Enc}(\text{pp}, \text{pk}, \text{M}) \rightarrow \text{C}$: The probabilistic encryption algorithm takes a public parameter pp, an encryption key pk, and a message M as input and outputs a ciphertext C.

$\text{Dec}(\text{pp}, \text{sk}, \text{C}) \rightarrow \text{M}/\perp$: The deterministic decryption algorithm takes a public parameter pp, a decryption key sk, and a ciphertext C as input and outputs a message $\text{M} \in \mathcal{M}$ or an error symbol $\perp \notin \mathcal{M}$.

$\text{Game}_{\text{PKE}, \mathcal{A}}^{N\text{-IND-CCA-C}}(\kappa)$	$\text{O}_{\text{Chall}}(\text{pk}, M_0, M_1)$	$\text{O}_{\text{Corr}}(\text{pk})$
1 : $L_{\text{PK}}, L_{\text{Chall}}, L_{\text{Corr}} \leftarrow \emptyset$ 2 : $b \leftarrow \{0, 1\}$ 3 : $\text{pp} \leftarrow \text{Setup}(1^\kappa)$ 4 : foreach $i \in [N]$ do 5 : $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp})$ 6 : $L_{\text{PK}} \leftarrow (\text{pk}_i, \text{sk}_i)$ 7 : $b' \leftarrow \mathcal{A}^{\text{O}_{\text{Dec}}, \text{O}_{\text{Corr}}, \text{O}_{\text{Chall}}}(\text{pp}, \{\text{pk}_i\}_{i \in [N]})$ 8 : if $L_{\text{Chall}} \cap L_{\text{Corr}} \neq \emptyset$ then 9 : return b 10 : if $b = b'$ then return 1 11 : else return 0	1 : if $(\text{pk}, *) \notin L_{\text{PK}} \vee (\text{pk}, *) \in L_{\text{Chall}}$ then 2 : return $\perp$ 3 : $C \leftarrow \text{Enc}(\text{pp}, \text{pk}, M_b)$ 4 : $L_{\text{Chall}} \leftarrow (\text{pk}, C)$ 5 : return C $\text{O}_{\text{Dec}}(\text{pk}, C)$ 1 : if $(\text{pk}, *) \notin L_{\text{PK}} \vee (\text{pk}, C) \in L_{\text{Chall}}$ then 2 : return $\perp$ 3 : $\text{sk} \leftarrow L_{\text{PK}}[\text{pk}]$ 4 : return $\text{Dec}(\text{sk}, C)$ $\text{O}'_{\text{Chall}}(\text{pk}_0, \text{pk}_1, M)$: $\mathcal{A}$ queries Q times 1 : if $(\text{pk}_0, *) \notin L_{\text{PK}} \vee (\text{pk}_1, *) \notin L_{\text{PK}}$ then 2 : return $\perp$ 3 : $C \leftarrow \text{Enc}(\text{pp}, \text{pk}_b, M)$ 4 : $L_{\text{Chall}} \leftarrow (\text{pk}_b, C)$ 5 : return C	1 : if $(\text{pk}, *) \notin L_{\text{PK}}$ then 2 : return $\perp$ 3 : $L_{\text{Corr}} \leftarrow \text{pk}$ 4 : $\text{sk} \leftarrow L_{\text{PK}}[\text{pk}]$ 5 : return sk

Figure 3: Security game for public-key encryption. $\mathcal{A}$ can send each pk_i^* to O_{Chall} only once.

We require a PKE scheme PKE to satisfy correctness and multi-user and single-challenge IND-CCA security with adaptive corruption (N -IND-CCA-C) introduced by Lee et al. [LLP20]. Note that 1-IND-CCA-C security is identical to the standard single-user and single-challenge IND-CCA security. Also, it is known that 1-IND-CCA-C (i.e., IND-CCA) security implies N -IND-CCA-C with the security loss of $O(N)$.

Definition 6 (Correctness). *We say that a PKE scheme PKE is correct if for all $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any key $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, and any message $M \in \mathcal{M}$, it holds that $\text{Dec}(\text{pp}, \text{sk}, \text{Enc}(\text{pp}, \text{pk}, M)) = M$ with overwhelming probability.*

Definition 7 (N -IND-CCA-C security). *Let PKE be a PKE scheme and $N = \text{poly}(\kappa)$. We define the game $\text{Game}_{\text{PKE}, \mathcal{A}}^{N\text{-IND-CCA-C}}$ in Figure 3 and the advantage of $\mathcal{A}$ as*

$$\text{Adv}_{\text{PKE}, \mathcal{A}}^{N\text{-IND-CCA-C}}(\kappa) := \left| \Pr \left[\text{Game}_{\text{PKE}, \mathcal{A}}^{N\text{-IND-CCA-C}}(\kappa) = 1 \right] - \frac{1}{2} \right|.$$

We say PKE is N -IND-CCA-C secure if $\text{Adv}_{\text{PKE}, \mathcal{A}}^{N\text{-IND-CCA-C}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

2.4 One-Time Signature

Definition 8 (One-Time Signature). *A one-time signature scheme OTS consists of the following three algorithms:*

$\text{KeyGen}(1^\kappa) \rightarrow (\text{ovk}, \text{osk})$: *The probabilistic key generation algorithm takes a security parameter 1^κ as input and outputs a pair of verification and signing keys (ovk, osk) .*

$\text{Sign}(\text{osk}, \text{omsg}) \rightarrow \text{osig}$: *The probabilistic signature generation algorithm takes a signing key osk and a message omsg as input and outputs a signature osig .*

$\text{Verify}(\text{ovk}, \text{omsg}, \text{osig}) \rightarrow 1/0$: The deterministic verification algorithm takes a verification key ovk , a message omsg and a signature osig as input and outputs 1 or 0.

We require an OTS scheme OTS to satisfy correctness and strong EUF-CMA (sEUF-CMA) security.

Definition 9 (Correctness). We say that an OTS scheme OTS is correct if for all $\kappa \in \mathbb{N}$, any key $(\text{ovk}, \text{osk}) \leftarrow \text{KeyGen}(1^\kappa)$, and any message $\text{omsg} \in \mathcal{M}$, it holds that $\text{Verify}(\text{ovk}, \text{omsg}, \text{Sign}(\text{osk}, \text{omsg})) = 1$ with overwhelming probability.

Definition 10 (sEUF-CMA security). Let OTS be an OTS scheme. We define the game $\text{Game}_{\text{OTS}, \mathcal{A}}^{\text{sEUF-CMA}}$ as

$\text{Game}_{\text{OTS}, \mathcal{A}}^{\text{sEUF-CMA}}(\kappa)$

```

1 :  $(\text{ovk}^*, \text{osk}^*) \leftarrow \text{KeyGen}(1^\kappa)$ 
2 :  $\text{omsg} \leftarrow \mathcal{A}(\text{ovk}^*)$ 
3 :  $\text{osig} \leftarrow \text{Sign}(\text{osk}^*, \text{omsg})$ 
4 :  $(\text{omsg}^*, \text{osig}^*) \leftarrow \mathcal{A}(\text{osig})$ 
5 : if  $(\text{omsg}^*, \text{osig}^*) \neq (\text{omsg}, \text{osig})$ 
       $\wedge \text{Verify}(\text{ovk}^*, \text{omsg}^*, \text{osig}^*) = 1$  then
6 :   return 1
7 : else return 0

```

and the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{OTS}, \mathcal{A}}^{\text{sEUF-CMA}}(\kappa) := \Pr \left[\text{Game}_{\text{OTS}, \mathcal{A}}^{\text{sEUF-CMA}}(\kappa) = 1 \right].$$

We say OTS is sEUF-CMA secure if $\text{Adv}_{\text{OTS}, \mathcal{A}}^{\text{sEUF-CMA}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

2.5 Non-Interactive Zero-Knowledge Argument

Let $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ be a polynomial time recognizable binary relation. For $(x, w) \in \mathcal{R}$, we call x the statement and w the witness. Let $\mathcal{L}$ be the corresponding NP language $\mathcal{L} = \{x \mid \exists w \text{ s.t. } (x, w) \in \mathcal{R}\}$. Below, we define non-interactive zero-knowledge arguments for NP languages.

Definition 11 (NIZK Arguments). A non-interactive zero-knowledge (NIZK) argument NIZK for the relation $\mathcal{R}$ consists of three algorithms.

$\text{Setup}(1^\kappa) \rightarrow \text{crs}$: The setup algorithm takes as input the security parameter 1^κ and outputs a common reference string crs .

$\text{Prove}(\text{crs}, x, w) \rightarrow \pi$: The prover's algorithm takes crs , a statement x and a witness w as input, and outputs a proof π .

$\text{Verify}(\text{crs}, x, \pi) \rightarrow 1/0$: The verifier's algorithm takes crs , x , and π as input, and outputs 1 or 0.

Definition 12 (Correctness). We say an NIZK argument NIZK is correct if for all pairs $(x, w) \in \mathcal{R}$, if we run $\text{crs} \leftarrow \text{Setup}(1^\kappa)$, then it holds that $\Pr[\text{Verify}(\text{crs}, x, \text{Prove}(\text{crs}, x, w)) = 1] = 1$.

Definition 13 (Soundness). We say an NIZK argument NIZK is sound if for any PPT adversary $\mathcal{A}$, if we run $\text{crs} \leftarrow \text{Setup}(1^\kappa)$ and $(x, \pi) \leftarrow \mathcal{A}(1^\kappa, \text{crs})$, then we have $\Pr[x \notin \mathcal{L} \wedge \text{Verify}(\text{crs}, x, \pi) = 1] \leq \text{negl}(\kappa)$.

Definition 14 (Zero-Knowledge). We say an NIZK argument NIZK is zero-knowledge if for any PPT adversary $\mathcal{A}$, there exists a PPT simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ such that if we run $\text{crs} \leftarrow \text{Setup}(1^\kappa)$ and $(\overline{\text{crs}}, \bar{\tau}) \leftarrow \text{Sim}_1(1^\kappa)$, then we have

$$\left| \Pr \left[\mathcal{A}^{\mathcal{O}_0(\text{crs}, \cdot, \cdot)}(1^\kappa, \text{crs}) = 1 \right] - \Pr \left[\mathcal{A}^{\mathcal{O}_1(\overline{\text{crs}}, \bar{\tau}, \cdot, \cdot)}(1^\kappa, \overline{\text{crs}}) = 1 \right] \right| \leq \text{negl}(\kappa),$$

where $\mathcal{O}_0(\text{crs}, x, w)$ outputs $\text{Prove}(\text{crs}, x, w)$ if $(x, w) \in \mathcal{R}$ and $\perp$ otherwise, and $\mathcal{O}_1(\overline{\text{crs}}, \bar{\tau}, x, w)$ outputs $\text{Sim}_2(\overline{\text{crs}}, \bar{\tau}, x)$ if $(x, w) \in \mathcal{R}$ and $\perp$ otherwise.

3 Comprehensive Definition of Multi-Designated Verifier Signatures

We reconsider the definitions of subset simulatable MDVS and provide a comprehensive framework. We believe the following definition will be helpful for future research on MDVS.

Before introducing our formal definition of MDVS, we explain the system overview of subset simulatable MDVS. In MDVS, there are two entities: signers and verifiers. They have their own secret and public key pairs; signers have signing key, and verifiers have verification key. When a signer generates a signature for a message, it selects *designated verifiers*, the set of verifiers who are allowed to verify the signature⁶. For example, in a group messaging system, users have their friends list, and when they send a message, they select the receivers from the list and consider the selected friends as the designated verifiers. Note that the designated verifiers can be changed for each message.

3.1 Syntax of MDVS

We now provide a formal syntax of MDVS, following the syntax presented by Damgård et al. [DHM⁺20]. MDVS consists of six algorithms (Setup, SKGen, VKGen, Sign, PrivVrfy, Sim). SKGen and VKGen generate the signer's and verifier's key pair, respectively. Sign generates a real signature for designated verifiers with a signer's secret key. Sim generates a simulated signature for designated verifiers with a designated verifier's secret key. PrivVrfy is used for signature verification.

Definition 15. A (subset simulatable) multi-designated verifier signature scheme MDVS consists of six algorithms.

$\text{Setup}(1^\kappa) \rightarrow \text{pp}$: The probabilistic setup algorithm takes a security parameter 1^κ as input and outputs a public parameter pp . Note that pp defines a message space $\mathcal{M}$.

$\text{SKGen}(\text{pp}) \rightarrow (\text{spk}, \text{ssk})$: The probabilistic signer key generation algorithm takes pp as input and outputs a pair of public and secret keys for a signer (spk, ssk) ⁷.

$\text{VKGen}(\text{pp}) \rightarrow (\text{vpk}, \text{vsk})$: The probabilistic verifier key generation algorithm takes pp as input and outputs a pair of public and secret keys for a verifier (vpk, vsk) ⁷.

$\text{Sign}(\text{pp}, \text{ssk}, \mathcal{D}, \text{m}) \rightarrow \sigma$: The probabilistic signing algorithm takes pp , a signer's secret key ssk , a designated set (a set of designated verifiers' public keys) $\mathcal{D} = \{\text{vpk}_j\}_{j \in [|\mathcal{D}|]}$, and a message m as input and outputs a signature σ .

$\text{PrivVrfy}(\text{pp}, \text{spk}, \text{vsk}, \mathcal{D}, \text{m}, \sigma) \rightarrow 1/0$: The deterministic private verification algorithm takes pp , a signer's public key spk , a verifier's secret key vsk , a designated set $\mathcal{D}$, a message m , and a signature σ as input and outputs 1 or 0.

$\text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m}) \rightarrow \sigma$: The probabilistic signature simulation algorithm takes pp , spk , a designated set $\mathcal{D}$, a set of verifiers' secret keys $\mathcal{C} = \{\text{vsk}_j\}_{j \in [|\mathcal{C}|]}$ ($0 \leq |\mathcal{C}| \leq |\mathcal{D}|$), and a message m as input and outputs a signature σ .

We say that MDVS is *privately verifiable* MDVS (PrivMDVS) if designated verifiers use their secret key to verify signatures. We introduce *publicly verifiable* MDVS (PubMDVS) that allows the verification of signatures without secret keys. In PubMDVS, we can see the signer specifies that the users are allowed to simulate signatures.

Definition 16. A publicly verifiable MDVS scheme PubMDVS is an MDVS scheme that uses the following verification algorithm instead of PrivVrfy.

⁶In the algorithm, the signer specifies the designated verifiers by using their verification key.

⁷Assume the secret key contains the corresponding public key.

$\text{PubVrfy}(\text{pp}, \text{spk}, \mathcal{D}, \text{m}, \sigma) \rightarrow 1/0$: The deterministic public verification algorithm takes pp , a signer's public key spk , a set of designated verifiers' public keys $\mathcal{D}$, a message m , and a signature σ as input and outputs 1 or 0.

We use MDVS instead of PrivMDVS or PubMDVS if we do not need to explicit verifiability.

3.2 Security Requirements for MDVS

MDVS is required to satisfy correctness, unforgeability, off-the-record (OTR) and optionally consistency. Roughly speaking, unforgeability states that valid signatures cannot be forged without knowing a signing key and OTR states that real signatures are indistinguishable from simulated signatures. Consistency says that an inconsistent signature that will be accepted by some designated verifiers and rejected by others cannot be created. These security notions for subset simulatable MDVS were provided by [DHM⁺20, CHMR23]. We follow their notion and take care of the conventional security notions in [ZAYS12].

Throughout this paper, we say that MDVS is (xxx, yyy)-secure, where xxx stands for the level of unforgeability (weak or strong) and yyy stands for the level of off-the-record (simDV, simDV+S, or allDV+S).

Correctness. Correctness says the designated verifiers accept signatures generated by a signer. It is defined as follows. Whether PrivVrfy or PubVrfy is used depends on the schemes.

Definition 17 (Correctness). We say that an MDVS scheme MDVS is correct if for any $\mu = \text{poly}(\kappa)$, any $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any signer key $(\text{spk}, \text{sk}) \leftarrow \text{SKGen}(\text{pp})$, all verifier keys $(\text{vpk}_i, \text{vsk}_i) \leftarrow \text{VKGen}(\text{pp})$ ($i \in [\mu]$), and any message $\text{m} \in \mathcal{M}$, it holds that

$$\begin{cases} \text{PrivVrfy}(\text{pp}, \text{spk}, \text{vsk}_i, \mathcal{D}, \text{m}, \sigma) = 1 & (\text{PrivMDVS}) \\ \text{PubVrfy}(\text{pp}, \text{spk}, \mathcal{D}, \text{m}, \sigma) = 1 & (\text{PubMDVS}) \end{cases}$$

with overwhelming probability, where $\mathcal{D} := \{\text{vpk}_j\}_{j \in [\mu]}$ and $\sigma \leftarrow \text{Sign}(\text{pp}, \text{sk}, \mathcal{D}, \text{m})$.

Consistency. Consistency, introduced by Damgård et al. [DHM⁺20], guarantees that uncorrupted verifiers receive the consistent verification result from the same signature, even if it is forged by an adversary who corrupts both the signer and the other verifiers. Note that consistency makes sense only for PrivMDVS since PubMDVS has consistency by default; anyone obtains the same result from $(\text{pp}, \text{spk}, \mathcal{D}, \text{m}, \sigma)$ with the deterministic algorithm PubVrfy. Below, we recall the formal definition presented by Damgård et al..

Definition 18 (Consistency). Let PrivMDVS be a PrivMDVS scheme. We define the game $\text{Game}_{\text{MDVS}, \mathcal{A}}^{\text{Con}}$ in Figure 5 and the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{PrivMDVS}, \mathcal{A}}^{\text{Con}}(\kappa) := \Pr \left[\text{Game}_{\text{PrivMDVS}, \mathcal{A}}^{\text{Con}}(\kappa) = 1 \right].$$

We say that PrivMDVS has consistency if $\text{Adv}_{\text{PrivMDVS}, \mathcal{A}}^{\text{Con}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

Remark 2 (On signing oracle in MDVS security games). To date, we have considered the case that the signing oracle in the MDVS security games only accepts honestly generated keys. We are not sure why such a formalization was done in the literature. Still, we guess that, in the real world, it is reasonable to assume that signing and verification keys are authorized by a trusted third party (e.g., CA) and anyone can check if they are honestly generated. In this case, we can assume signatures are generated only for honestly generated keys.

Game_{MDVS, A}^{xxx-Unf}(κ): $xxx \in \{\text{weak}, \text{strong}\}$ <pre> 1 : $L_{SPK}, L_{SSK}, L_{VPK}, L_{VSK}, L_{Sig}, L_{Ver} \leftarrow \emptyset$ 2 : $pp \leftarrow \text{Setup}(1^\kappa)$ 3 : $(spk^*, \mathcal{D}^*, m^*, \sigma^*) \leftarrow \mathcal{A}^{O_{SPK}, O_{SSK}, O_{VPK}, O_{VSK}, O_{Sig}, O_{Ver}}(pp)$ 4 : if $(spk^*, ssk^*) \in L_{SPK} \wedge spk^* \notin L_{SSK}$ $\wedge \forall vpk \in \mathcal{D}^* : (vpk, vsk) \in L_{VPK} \wedge vpk \notin L_{VSK}$ $\wedge (spk^*, \mathcal{D}^*, m^*) \notin L_{Sig}$ // When MDVS is privately verifiable $\wedge \exists vpk^* \in \mathcal{D}^* : (vpk^*, vsk^*) \in L_{VPK} \wedge vpk^* \notin L_{VSK}$ $\wedge \text{PrivVrfy}(pp, spk^*, vsk^*, \mathcal{D}^*, m^*, \sigma^*) = 1$ // When MDVS is publicly verifiable $\wedge \text{PubVrfy}(pp, spk^*, \mathcal{D}^*, m^*, \sigma^*) = 1$ then 5 : return 1 6 : else 7 : return 0 </pre>	O_{SPK}($\cdot$) <pre> 1 : $(spk, ssk) \leftarrow \text{SKGen}(pp)$ 2 : $L_{SPK} \leftarrow (spk, ssk)$ 3 : return spk </pre>
	O_{SSK}(spk) <pre> 1 : if $(spk, *) \notin L_{SPK}$ then 2 : return $\perp$ 3 : $L_{SSK} \leftarrow spk$ 4 : $ssk \leftarrow L_{SPK}[spk]$ 5 : return ssk </pre>
	O_{VPK}($\cdot$) <pre> 1 : $(vpk, vsk) \leftarrow \text{VKGen}(pp)$ 2 : $L_{VPK} \leftarrow (vpk, vsk)$ 3 : return vpk </pre>
Game_{MDVS, A}^{yyy-OTR}(κ): $yyy \in \{\text{simDV}, \text{simDV+S}, \text{allDV+S}\}$ <pre> 1 : $L_{SPK}, L_{SSK}, L_{VPK}, L_{VSK}, L_{Sig}, L_{Ver} \leftarrow \emptyset$ 2 : $b \leftarrow \{0, 1\}$ 3 : $pp \leftarrow \text{Setup}(1^\kappa)$ 4 : $(spk^*, \mathcal{D}^*, \mathcal{C}^*, m^*) \leftarrow \mathcal{A}^{O_{SPK}, O_{SSK}, O_{VPK}, O_{VSK}, O_{Sig}, O_{Ver}}(pp)$ 5 : $\mathcal{C}_{vpk}^* \leftarrow \{vpk : vsk \in \mathcal{C}^* \wedge (vpk, vsk) \in L_{VPK}\}$ 6 : if $(spk^*, ssk^*) \in L_{SPK} \wedge (\forall vpk \in \mathcal{D}^* : (vpk, vsk) \in L_{VPK})$ $\wedge \mathcal{C}_{vpk}^* = \mathcal{C}^* \wedge \mathcal{C}_{vpk}^* \subseteq \mathcal{D}^*$ then 7 : continue 8 : else return b 9 : $ssk^* \leftarrow L_{SPK}[spk^*]$ 10 : $\sigma_0^* \leftarrow \text{Sim}(pp, spk^*, \mathcal{D}^*, \mathcal{C}^*, m^*)$ 11 : $\sigma_1^* \leftarrow \text{Sign}(pp, ssk^*, \mathcal{D}^*, m^*)$ 12 : $b' \leftarrow \mathcal{A}^{O_{SPK}, O_{SSK}, O_{VPK}, O_{VSK}, O_{Sig}, O_{Ver}}(\sigma_b^*)$ 13 : if $spk^* \notin L_{SSK}$ $\wedge \forall vpk \in \mathcal{D}^* \setminus \mathcal{C}_{vpk}^* : vpk \notin L_{VSK}$ // When MDVS is privately verifiable, $\mathcal{A}$ must not verify σ_b via O_{Ver}. $\wedge \forall vpk^* \in \mathcal{D}^* \setminus \mathcal{C}_{vpk}^* : (spk^*, vpk^*, \mathcal{D}^*, m^*, \sigma_b) \notin L_{Ver}$ then 14 : continue 15 : else return b 16 : if $b = b'$ then 17 : return 1 18 : else 19 : return 0 </pre>	O_{VSK}(vpk) <pre> 1 : if $(vpk, *) \notin L_{VPK}$ then 2 : return $\perp$ 3 : $L_{VSK} \leftarrow vpk$ 4 : $vsk \leftarrow L_{VPK}[vpk]$ 5 : return vsk </pre>
	O_{Sig}($spk, \mathcal{D}, m$) <pre> 1 : if $(spk, *) \notin L_{SPK}$ then 2 : return $\perp$ 3 : if $\exists vpk \in \mathcal{D} : (vpk, vsk) \notin L_{VPK}$ then 4 : return $\perp$ 5 : $ssk \leftarrow L_{SPK}[spk]$ 6 : $\sigma \leftarrow \text{Sign}(pp, ssk, \mathcal{D}, m)$ 7 : $L_{Sig} \leftarrow (spk, \mathcal{D}, m)$ 8 : return σ </pre>
	O_{Ver}($spk, vpk, \mathcal{D}, m, \sigma$) <pre> 1 : if $(spk, *) \notin L_{SPK} \vee (vpk, *) \notin L_{VPK}$ then 2 : return $\perp$ 3 : $vsk \leftarrow L_{VPK}[vpk]$ 4 : $b \leftarrow \text{PrivVrfy}(pp, spk, vsk, \mathcal{D}, m, \sigma)$ 5 : $L_{Ver} \leftarrow (spk, vpk, \mathcal{D}, m, \sigma)$ 6 : return b </pre>

Figure 4: Security games for multi-designated verifier signatures. If the MDVS scheme is publicly verifiable, the O_{Ver} oracle is not provided. The grayed/boxed codes are only for the corresponding security games.

$\text{Game}_{\text{PrivMDVS}, \mathcal{A}}^{\text{Con}}(\kappa)$	
1 :	$L_{\text{SPK}}, L_{\text{SSK}}, L_{\text{VPK}}, L_{\text{VSK}}, L_{\text{Sig}}, L_{\text{Ver}} \leftarrow \emptyset$
2 :	$\text{pp} \leftarrow \text{Setup}(1^\kappa)$
3 :	$(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^*) \leftarrow \mathcal{A}^{\text{O}_{\text{SPK}}, \text{O}_{\text{SSK}}, \text{O}_{\text{VPK}}, \text{O}_{\text{VSK}}, \text{O}_{\text{Sig}}, \text{O}_{\text{Ver}}}(\text{pp})$
4 :	if $(\text{spk}^*, \text{ssk}^*) \in L_{\text{SPK}} \wedge \forall \text{vpk} \in \mathcal{D}^* : (\text{vpk}, \text{vsk}) \in L_{\text{VPK}}$ $\wedge \exists \text{vpk}_0^*, \text{vpk}_1^* \in \mathcal{D}^* :$ $\text{vpk}_0^* \neq \text{vpk}_1^* \wedge \text{vpk}_0^* \notin L_{\text{VSK}} \wedge \text{vpk}_1^* \notin L_{\text{VSK}}$ $\wedge \text{PrivVrfy}(\text{pp}, \text{spk}^*, \text{vsk}_0^*, \mathcal{D}^*, \text{m}^*, \sigma^*) = 0$ $\wedge \text{PrivVrfy}(\text{pp}, \text{spk}^*, \text{vsk}_1^*, \mathcal{D}^*, \text{m}^*, \sigma^*) = 1$ then
5 :	return 1
6 :	else return 0

Figure 5: Security games for multi-designated verifier signatures (continuation from Figure 4).

Unforgeability. Unforgeability guarantees that no valid signature can be generated that is accepted by an uncorrupted designated verifier without knowing the signer’s and the verifier’s secret keys. According to [DHM⁺20], we have two flavors, *weak* and *strong* unforgeability, depending on whether an adversary has permission to run the simulator by itself.

- Weak unforgeability (**weak-Unf**) [ZAYS12]: An adversary is not allowed to run the simulator. In subset simulatable MDVS, the adversary cannot corrupt any designated verifiers linked to the forged signature because knowing one of the designated verifier’s secrets is sufficient to run the simulator.
- Strong unforgeability (**strong-Unf**) [DHM⁺20, CHMR23]: An adversary is allowed to run the simulator, i.e., it is allowed to corrupt designated verifiers.

Unforgeability is formally defined as follows.

Definition 19 (Unforgeability). *Let MDVS be an MDVS scheme and $\text{xxx} \in \{\text{weak}, \text{strong}\}$. We define the game $\text{Game}_{\text{MDVS}, \mathcal{A}}^{\text{xxx-Unf}}$ in Figure 4 and the advantage of $\mathcal{A}$ as*

$$\text{Adv}_{\text{MDVS}, \mathcal{A}}^{\text{xxx-Unf}}(\kappa) := \Pr \left[\text{Game}_{\text{MDVS}, \mathcal{A}}^{\text{xxx-Unf}}(\kappa) = 1 \right].$$

We say that MDVS is xxx-Unf secure if $\text{Adv}_{\text{MDVS}, \mathcal{A}}^{\text{xxx-Unf}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

Remark 3 (Verification oracle in the unforgeability game). Damgård et al. [DHM⁺20] did not provide the verification oracle to the adversary in the unforgeability game. Later, Chakraborty et al. [CHMR23] pointed out a verification oracle is necessary for constructive modeling of MDVS [MPR21] and high-layer primitives based on MDVS [MPR22] and updated the unforgeability game so that the adversary can access the verification oracle.

Remark 4 (Note on unforgeability in *all-simulatable* MDVS). All-simulatable MDVS [ZAYS12] requires all of the designated verifier’s secrets to simulate signatures. Thus, even if the adversary corrupts some designated verifiers, it cannot execute the simulator. Also, corrupting all verifiers is a meaningless attack since there are no uncorrupted verifiers. Therefore, all-simulatable MDVS can only achieve weak unforgeability.

Off-The-Record (OTR). OTR guarantees simulated “fake” signatures are indistinguishable from the signer’s “real” signatures. According to the existing works [ZAYS12, DHM⁺20, CHMR23], we have three flavors of OTR, depending on what type of keys the adversary can use to distinguish signatures. (This work gives each abbreviation.)

- Secret keys of designated verifiers in simulation (simDV) [DHM⁺20]: Even if an adversary gets the secret keys of designated verifiers who are involved in the simulation, simulated signatures are indistinguishable from real ones;
- Secret keys of designated verifiers in simulation and a signer (simDV+S) [CHMR23]: Even if an adversary gets the secret keys of designated verifiers who are involved in the simulation and a signer, simulated signatures are indistinguishable from real ones;
- Secret keys of all designated verifiers and a signer (allDV+S) [ZAYS12]: Even if an adversary gets the secret keys of all designated verifiers and a signer, simulated signatures are indistinguishable from the real ones.

OTR is formally defined as follows.

Definition 20 (Off-the-record (OTR)). *Let $yyy \in \{\text{simDV}, \text{simDV}+\text{S}, \text{allDV}+\text{S}\}$ and MDVS be an MDVS scheme. We define the game $\text{Game}_{\text{MDVS}, \mathcal{A}}^{yyy\text{-OTR}}$ in Figure 4 and the advantage of $\mathcal{A}$ as*

$$\text{Adv}_{\text{MDVS}, \mathcal{A}}^{yyy\text{-OTR}}(\kappa) := \left| \Pr \left[\text{Game}_{\text{MDVS}, \mathcal{A}}^{yyy\text{-OTR}}(\kappa) = 1 \right] - \frac{1}{2} \right|.$$

We say that MDVS is $yyy\text{-OTR}$ if $\text{Adv}_{\text{MDVS}, \mathcal{A}}^{yyy\text{-OTR}}(\kappa) \leq \text{negl}(\kappa)$ for any PPT adversary $\mathcal{A}$.

Remark 5 (Restriction for verification queries). A verification oracle is provided in the OTR game since adversaries cannot verify signatures by themselves. Specifically, in the game, adversaries are constrained from sending the challenge signature to the oracle to prevent trivial attacks. It is noteworthy that this restriction has varied in existing works. Damgård et al. [DHM⁺20] imposed a stringent prohibition, disallowing adversaries from sending the challenge signature σ_b^* to O_{Ver} . However, this restriction is deemed overly strong, preventing the adversary from verifying σ_b^* with unrelated signers, verifiers, or messages. Ideally, the adversary should be unable to distinguish the signature, even if σ_b^* is verified with unrelated keys and messages. The subsequent work [CHMR23] relaxed this restriction, allowing such queries, but it still restricts the verification of σ_b^* with corrupted (i.e., simulating) verifiers. This restriction proves futile since the adversary possesses the secret keys of corrupted verifiers, enabling the adversary to verify σ_b^* . We now only prohibit verifying the challenge signature with uncorrupted verifiers to align the restriction with an adversary’s capabilities.

Remark 6 (Existing works). We remark that our framework encompasses the existing subset simulatable MDVS schemes [DHM⁺20, CHMR23]. Regarding unforgeability, both schemes accept the corruption of designated verifiers, thus they are strongly unforgeable. Note that as mentioned in Remark 3, [DHM⁺20] does not provide verification oracle in the unforgeability game. In [DHM⁺20], the adversary in the OTR game is not allowed to learn the signer’s signing key. That is, it is (strong, simDV)-PrivMDVS in our terminology. On the other hand, the scheme in [CHMR23] was proposed to allow the “signing key leakage” in the OTR game (i.e., the corruption of the signer’s signing key). However, it is still prohibited to corrupt all the designated verifiers in the game. Therefore, it is (strong, simDV+S)-PrivMDVS in our framework. Note that as explained in Remark 5, there is a difference on the restriction for verification queries. Also, both of the schemes achieve consistency.

4 Impossibility Results on MDVS

In this section, we show (strong, allDV+S)-PrivMDVS and strong-Unf and any OTR secure PubMDVS are impossible. To this end, we show the relationship between the property of simulated signatures and the security properties. Then, we show the contradiction in the simulated signatures, indicating the impossibility results.

Properties of simulated signatures. The security properties of MDVS implicitly define the possible results when the verification algorithm verifies honestly-generated simulated signatures. Here, we explicitly specify the property of the simulated signatures of each MDVS scheme.

We first show that **strong-Unf** secure MDVS scheme produces simulated signatures that allow designated verifiers to detect it is a simulated signature.

Theorem 7. *If a PrivMDVS scheme PrivMDVS is strong-Unf secure, then it satisfies the following property: for any $\mu, \nu = \text{poly}(\kappa)$ such that $1 \leq \nu \leq \mu$, any $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any signer key $(\text{spk}, \text{spk}) \leftarrow \text{SKGen}(\text{pp})$, all verifier keys $(\text{vpk}_i, \text{vpk}_i) \leftarrow \text{VKGen}(\text{pp})$ ($i \in [\mu]$), any $\mathcal{C} := \{\text{vsk}_j\}_{j \in [\nu]} \subseteq \{\text{vsk}_i\}_{i \in [\mu]}$, and any message $\mathbf{m} \in \mathcal{M}$, it holds that $\text{PrivVrfy}(\text{pp}, \text{spk}, \text{vsk}, \mathcal{D}, \mathbf{m}, \sigma) = 0$ for $\text{vsk} \in \{\text{vsk}_i\}_{i \in [\mu]} \setminus \mathcal{C}$ with overwhelming probability, where $\mathcal{D} := \{\text{vpk}_i\}_{i \in [\mu]}$, and $\sigma \leftarrow \text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \mathbf{m})$.*

Proof. Let us consider the following adversary $\mathcal{A}_{m,C}$ parameterized by an integer $m \in \mathbb{N}$ and a set $C \subset [m]$:

1. Upon receiving pp , $\mathcal{A}_{m,C}$ obtains a signer's public key vpk^* , m verifier's public keys $\{\text{vpk}_i\}_{i \in [m]}$ and verifier's secret keys $\{\text{vsk}_i\}_{i \in C}$ by using its oracle.
2. $\mathcal{A}_{m,C}$ samples $\mathbf{m}^* \leftarrow \mathcal{M}$ and sets $\mathcal{D}^* := \{\text{vpk}_i\}_{i \in [m]}$ and $\mathcal{C}^* := \{\text{vsk}_j\}_{j \in C}$.
3. $\mathcal{A}_{m,C}$ generates $\sigma^* \leftarrow \text{Sim}(\text{pp}, \text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \mathbf{m}^*)$.
4. $\mathcal{A}_{m,C}$ outputs $(\text{spk}^*, \mathcal{D}^*, \mathbf{m}^*, \sigma^*)$ as a forgery.

As observed, $\mathcal{A}_{m,C}$ does not know ssk^* and verifier's secret keys vsk_j such that $j \in [m] \setminus C$. Also, $\mathcal{A}_{m,C}$ does not use OSig oracle. Thus, $\mathcal{A}_{m,C}$ is a valid adversary against strong-Unf security. On the other hand, since PrivMDVS is strong-Unf secure, $\mathcal{A}_{m,C}$ must have a negligible advantage. This means that $\text{PrivVrfy}(\text{pp}, \text{spk}^*, \text{vsk}_j, \mathcal{D}^*, \mathbf{m}^*, \sigma^*) = 0$ holds for vsk_j such that $j \in [m] \setminus C$. This concludes the proof. $\square$

We can obtain the following result about strong-Unf secure PubMDVS using the same argument above.

Theorem 8. *If a PubMDVS scheme PubMDVS is strong-Unf secure, then it satisfies the following property: for any $\mu, \nu = \text{poly}(\kappa)$ such that $1 \leq \nu \leq \mu$, any $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any signer key $(\text{spk}, \text{spk}) \leftarrow \text{SKGen}(\text{pp})$, all verifier keys $(\text{vpk}_i, \text{vpk}_i) \leftarrow \text{VKGen}(\text{pp})$ ($i \in [\mu]$), any $\mathcal{C} := \{\text{vsk}_j\}_{j \in [\nu]} \subseteq \{\text{vsk}_i\}_{i \in [\mu]}$, and any message $\mathbf{m} \in \mathcal{M}$, it holds that $\text{PubVrfy}(\text{pp}, \text{spk}, \mathcal{D}, \mathbf{m}, \sigma) = 0$ with overwhelming probability, where $\mathcal{D} := \{\text{vpk}_i\}_{i \in [\mu]}$, and $\sigma \leftarrow \text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \mathbf{m})$.*

Next, we show the possible verification result when simulated signatures of the OTR secure MDVS scheme are honestly verified with PrivVrfy/PubVrfy algorithm.

Theorem 9. *If a PrivMDVS scheme PrivMDVS is correct and either simDV+S-OTR or simDV-OTR secure, then it satisfies the following property: for any $\mu, \nu = \text{poly}(\kappa)$ such that $1 \leq \nu \leq \mu$, any $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any signer key $(\text{spk}, \text{spk}) \leftarrow \text{SKGen}(\text{pp})$, all verifier keys $(\text{vpk}_i, \text{vpk}_i) \leftarrow \text{VKGen}(\text{pp})$ ($i \in [\mu]$), any $\mathcal{C} := \{\text{vsk}_j\}_{j \in [\nu]} \subseteq \{\text{vsk}_i\}_{i \in [\mu]}$, and any message $\mathbf{m} \in \mathcal{M}$, it holds that $\text{PrivVrfy}(\text{pp}, \text{spk}, \text{vsk}, \mathcal{D}, \mathbf{m}, \sigma) = 1$ for $\text{vsk} \in \mathcal{C}$ with overwhelming probability, where $\mathcal{D} := \{\text{vpk}_i\}_{i \in [\mu]}$, and $\sigma \leftarrow \text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \mathbf{m})$.*

Proof. We consider the following adversary $\mathcal{A}_{m,C}$ parameterized by $m \in \mathbb{N}$ and $C \subset [m]$:

1. Upon receiving $\mathcal{A}_{m,C}$, $\mathcal{A}$ obtains a signer's public key spk^* , m verifier's public keys $\{\text{vpk}_i\}_{i \in [m]}$ and verifier's secret keys $\{\text{vsk}_i\}_{i \in C}$ by using its oracle.
2. $\mathcal{A}_{m,C}$ sets $\mathcal{D}^* := \{\text{vpk}_i\}_{i \in [m]}$ and $\mathcal{C}^* := \{\text{vpk}_j\}_{j \in C}$ and samples $\mathbf{m}^* \leftarrow \mathcal{M}$.
3. $\mathcal{A}_{m,C}$ sends $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \mathbf{m}^*)$ and receives σ^* .
4. $\mathcal{A}_{m,C}$ outputs $b' \leftarrow \bigcap_{i \in C} \text{PrivVrfy}(\text{pp}, \text{spk}^*, \text{vsk}_i, \mathcal{D}^*, \mathbf{m}^*, \sigma^*)$.

As observed, $\mathcal{A}_{m,C}$ is a valid adversary against $\text{simDV}+\text{S-OTR}/\text{simDV-OTR}$ security. When the challenge bit is $b = 1$, the challenge signature is generated by Sign . In this case, $b' = 1$ with overwhelming probability due to correctness. On the other hand, when $b = 0$, σ^* is generated by the simulation algorithm Sim . In this case, since PrivMDVS is $\text{simDV}+\text{S-OTR}/\text{simDV-OTR}$ secure, the advantage of $\mathcal{A}_{m,C}$ must be negligible. This means that $b' = 1$ must hold, i.e., $\text{PrivVrfy}(\text{pp}, \text{spk}^*, \text{vsk}_i, \mathcal{D}^*, \text{m}^*, \sigma^*) = 1$ for all vsk_i ($i \in C$) with overwhelming probability. Since the above argument holds for any $\mathcal{A}_{m,C}$, this concludes the proof. $\square$

From the same argument as Theorem 9, we can also obtain the following result about $\text{allDV}+\text{S-OTR}$ secure PrivMDVS and OTR secure PubMDVS . Due to the page limitation, we defer the proofs to the full version of this paper.

Theorem 10. *If a PrivMDVS scheme PrivMDVS is correct and $\text{allDV}+\text{S-OTR}$ secure, then it satisfies the following property: for any $\mu, \nu = \text{poly}(\kappa)$ such that $1 \leq \nu \leq \mu$, any $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any signer key $(\text{spk}, \text{spk}) \leftarrow \text{SKGen}(\text{pp})$, all verifier keys $(\text{vpk}_i, \text{vpk}_i) \leftarrow \text{VKGen}(\text{pp})$ ($i \in [\mu]$), any $\mathcal{C} := \{\text{vsk}_j\}_{j \in [\nu]} \subseteq \{\text{vsk}_i\}_{i \in [\mu]}$, and any message $\text{m} \in \mathcal{M}$, it holds that $\text{PrivVrfy}(\text{pp}, \text{spk}, \text{vsk}, \mathcal{D}, \text{m}, \sigma) = 1$ for $\text{vsk} \in \{\text{vsk}_i\}_{i \in [\mu]}$ with overwhelming probability, where $\mathcal{D} := \{\text{vpk}_i\}_{i \in [\mu]}$, and $\sigma \leftarrow \text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m})$.*

Theorem 11. *Let $\text{yyy} \in \{\text{simDV}, \text{simDV}+\text{S}, \text{allDV}+\text{S}\}$. If a PubMDVS scheme PubMDVS is correct and yyy-OTR secure, then it satisfies the following property: for any $\mu, \nu = \text{poly}(\kappa)$ such that $1 \leq \nu \leq \mu$, any $\text{pp} \leftarrow \text{Setup}(1^\kappa)$, any signer key $(\text{spk}, \text{spk}) \leftarrow \text{SKGen}(\text{pp})$, all verifier keys $(\text{vpk}_i, \text{vpk}_i) \leftarrow \text{VKGen}(\text{pp})$ ($i \in [\mu]$), any $\mathcal{C} := \{\text{vsk}_j\}_{j \in [\nu]} \subseteq \{\text{vsk}_i\}_{i \in [\mu]}$, and any message $\text{m} \in \mathcal{M}$, it holds that $\text{PubVrfy}(\text{pp}, \text{spk}, \mathcal{D}, \text{m}, \sigma) = 1$ with overwhelming probability, where $\mathcal{D} := \{\text{vpk}_i\}_{i \in [\mu]}$, and $\sigma \leftarrow \text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m})$.*

Impossibility results. We conclude the following impossibility results as a consequence of the above theorems.

Theorem 12. *(strong, $\text{allDV}+\text{S}$)- PrivMDVS is impossible.*

Proof. Theorem 7 and Theorem 10 show the contradiction about the property of simulated signatures. Thus, (strong, $\text{allDV}+\text{S}$)- PrivMDVS is impossible. $\square$

Theorem 13. *(strong, yyy)- PubMDVS is impossible where $\text{yyy} \in \{\text{simDV}, \text{simDV}+\text{S}, \text{allDV}+\text{S}\}$.*

Proof. Theorem 8 and Theorem 11 show the contradiction about the property of simulated signatures. Thus, (strong, yyy)- PubMDVS is impossible for $\text{yyy} \in \{\text{simDV}, \text{simDV}+\text{S}, \text{allDV}+\text{S}\}$. $\square$

5 (strong, $\text{simDV}+\text{S}$)- PrivMDVS Implies IND-CCA Secure PKE

This section reveals that (strong, $\text{simDV}+\text{S}$)- PrivMDVS implies IND-CCA secure PKE. We first provide a generic construction of PKE from PrivMDVS .

Construction 1. Let $\text{PrivMDVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{PrivVrfy}, \text{Sim})$ be a PrivMDVS scheme. The PKE scheme Construction 1 is as follows.

PKE.Setup(1^κ): Compute $\text{pp}_{\text{MDVS}} \leftarrow \text{PrivMDVS.Setup}(1^\kappa)$, $(\text{spk}, \text{ssk}) \leftarrow \text{PrivMDVS.SKGen}(\text{pp}_{\text{MDVS}})$, and $(\text{vpk}, \text{vsk}) \leftarrow \text{PrivMDVS.VKGen}(\text{pp}_{\text{MDVS}})$ and sample $\text{m} \leftarrow \mathcal{M}_{\text{MDVS}}$. Output $\text{pp}_{\text{PKE}} := (\text{pp}_{\text{MDVS}}, \text{spk}, \text{ssk}, \text{vpk}, \text{vsk}, \text{m})$. Note that the message space is $\mathcal{M}_{\text{PKE}} = \{0, 1\}$.

PKE.KeyGen(pp_{PKE}): Compute $(\text{vpk}', \text{vsk}') \leftarrow \text{PrivMDVS.VKGen}(\text{pp}_{\text{MDVS}})$ and output $(\text{pk}, \text{sk}) := (\text{vpk}', \text{vsk}')$.

PKE.Enc($\text{pp}_{\text{PKE}}, \text{pk}, \text{M} \in \{0, 1\}$): If $\text{M} = 1$, compute $\sigma \leftarrow \text{PrivMDVS.Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}, \{\text{vpk}, \text{pk}\}, \text{m})$; Else, compute $\sigma \leftarrow \text{PrivMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \{\text{vpk}, \text{pk}\}, \{\text{vsk}\}, \text{m})$. Output $\text{C} := \sigma$.

PKE.Dec($\text{pp}_{\text{PKE}}, \text{sk}, \text{C}$) : Output $\text{M} \leftarrow \text{PrivMDVS.PrivVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}, \text{sk}, \{\text{vpk}, \text{pk}\}, \text{m}, \text{C})$.

We prove the correctness and IND-CCA security of Construction 1.

Theorem 14. *If PrivMDVS is correct and strong-Unf secure, Construction 1 is correct.*

Proof. Let us consider the case when $M = 1$ and $M = 0$ separately.

- If $M = 1$, the signature is generated by Sign. Due to the correctness of PrivMDVS, $\text{PrivVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}, \text{sk}, \{\text{vpk}, \text{pk}\}, \text{m}, \sigma) = 1$ holds with overwhelming probability.
- If $M = 0$, the signature is generated by Sim with $\mathcal{C} = \{\text{vsk}\}$ (i.e., $\text{sk} \notin \mathcal{C}$). Since PrivMDVS is strong-Unf secure, Theorem 7 implies that $\text{PrivVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}, \text{sk}, \{\text{vpk}, \text{pk}\}, \text{m}, \sigma) = 0$ holds with overwhelming probability.

Therefore, Construction 1 is correct. $\square$

Theorem 15. *If PrivMDVS is simDV+S-OTR secure, Construction 1 is IND-CCA secure.*

Proof. Let $\mathcal{A}$ be a PPT adversary that breaks the IND-CCA security of Construction 1.

We show a PPT adversary $\mathcal{B}$ that breaks the simDV+S-OTR security of PrivMDVS by using $\mathcal{A}$, as follows. Below, let d, d' be the challenge bit of IND-CCA game and let b, b' be the challenge bit of simDV+S-OTR game. Note that we set $b = d$ while constructing $\mathcal{B}$.

1. Upon receiving pp_{MDVS} , $\mathcal{B}$ generates (spk, ssk) , (vpk, vsk) , and $\text{pk} := \text{vpk}'$ with its key generation oracles and samples $\text{m} \leftarrow \mathcal{M}_{\text{MDVS}}$. Then, $\mathcal{B}$ executes $\mathcal{A}$ on input $\text{pp}_{\text{PKE}} := (\text{pp}_{\text{MDVS}}, \text{spk}, \text{ssk}, \text{vpk}, \text{vsk}, \text{m})$ and pk .
2. When $\mathcal{A}$ sends $C = \sigma$ to the O_{Dec} oracle, $\mathcal{B}$ sends $(\text{spk}, \text{vpk}, \{\text{vpk}, \text{pk}\}, \text{m}, \sigma)$ to its O_{Ver} oracle and receives M . Then, $\mathcal{B}$ returns M to $\mathcal{A}$. Note that after $\mathcal{A}$ receives the challenge C^* , it must be $C \neq C^*$.
3. $\mathcal{A}$ outputs (M_0, M_1) . Then, $\mathcal{B}$ works as follows.
 - If $(M_0, M_1) = (0, 0)$, $\mathcal{B}$ returns $\sigma^* \leftarrow \text{PrivMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \{\text{vpk}, \text{pk}\}, \{\text{vsk}\}, \text{m})$.
 - If $(M_0, M_1) = (1, 1)$, $\mathcal{B}$ returns $\sigma^* \leftarrow \text{PrivMDVS.Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}, \{\text{vpk}, \text{pk}\}, \text{m})$.
 - Else, $\mathcal{B}$ outputs $(\text{spk}, \{\text{vpk}, \text{pk}\}, \text{pk}, \text{m})$ to request the challenge signature σ^* and returns it.
4. Finally, $\mathcal{A}$ outputs a bit d' . Then, $\mathcal{B}$ works as follows.
 - If $(M_0, M_1) = (1, 0)$, $\mathcal{B}$ outputs $b' = d' \oplus 1$.
 - Else, $\mathcal{B}$ outputs $b' = d'$ as it is.

When $(M_0, M_1) = (0, 0)$ or $(M_0, M_1) = (1, 1)$, the challenge ciphertext $C^* = \sigma^*$ is independent of b ; Otherwise, if $b = d = 0$, C^* is an encryption of 0, else if $b = d = 1$, C^* is an encryption of 1. Therefore, if $(M_0, M_1) = (0, 1)$, $\mathcal{A}$ outputs $d = d'$ is the same as the probability $\mathcal{B}$ outputs $b = b'$. On the other hand, when $(M_0, M_1) = (1, 0)$, $\mathcal{A}$ outputs $d' = 0$ means it guesses C^* is an encryption of 1. Thus, the probability $\mathcal{A}$ outputs $d = d'$ is the same as the probability $\mathcal{B}$ outputs $b = b' \oplus 1$. Therefore, if $\mathcal{A}$ correctly guesses the challenge bit d in the IND-CCA game, $\mathcal{B}$ correctly guesses the challenge bit b in the simDV+S-OTR game. Thus, we have

$$\begin{aligned}
\text{Adv}_{\text{Const. 1}, \mathcal{A}}^{\text{IND-CCA}}(\kappa) &= \left| \Pr \left[\text{Game}_{\text{PKE}, \mathcal{A}}^{\text{IND-CCA}}(\kappa) = 1 \right] - \frac{1}{2} \right| \\
&= \left| \Pr[d = d'] - \frac{1}{2} \right| \\
&= \frac{1}{2} |\Pr[d' = 0 | d = 0] - \Pr[d' = 0 | d = 1]| \\
&= \frac{1}{2} |\Pr[d' = 0 \wedge (M_0, M_1) = (0, 0) | d = 0] - \Pr[d' = 0 \wedge (M_0, M_1) = (0, 0) | d = 1]|
\end{aligned}$$

$$\begin{aligned}
& + \Pr[d' = 0 \wedge (M_0, M_1) = (1, 1) | d = 0] - \Pr[d' = 0 \wedge (M_0, M_1) = (1, 1) | d = 1] \\
& + \Pr[d' = 0 \wedge (M_0, M_1) = (1, 0) | d = 0] - \Pr[d' = 0 \wedge (M_0, M_1) = (1, 0) | d = 1] \\
& + \Pr[d' = 0 \wedge (M_0, M_1) = (0, 1) | d = 0] - \Pr[d' = 0 \wedge (M_0, M_1) = (0, 1) | d = 1] \\
& = \frac{1}{2} |\Pr[d' = 0 \wedge (M_0, M_1) = (1, 0) | d = 0] - \Pr[d' = 0 \wedge (M_0, M_1) = (1, 0) | d = 1]| \\
& \quad + \Pr[d' = 0 \wedge (M_0, M_1) = (0, 1) | d = 0] - \Pr[d' = 0 \wedge (M_0, M_1) = (0, 1) | d = 1]| \\
& \leq \left| \Pr[b = b' \wedge (M_0, M_1) = (1, 0)] - \frac{1}{2} \right| + \left| \Pr[b = b' \wedge (M_0, M_1) = (0, 1)] - \frac{1}{2} \right| \\
& = 2\text{Adv}_{\text{PrivMDVS}, \mathcal{B}}^{\text{simDV+S-OTR}}(\kappa)
\end{aligned}$$

Since PrivMDVS is simDV+S-OTR secure, we have $\text{Adv}_{\text{Const. 1}, \mathcal{A}}^{\text{IND-CCA}}(\kappa) \leq \text{negl}(\kappa)$. Thus, Construction 1 is IND-CCA secure. $\square$

6 (weak, allDV+S)-PubMDVS is Equivalent to Ring Signature

This section shows the equivalence of (weak, allDV+S)-PubMDVS and ring signature (RS). We provide a generic construction of RS from (weak, allDV+S)-PubMDVS and vice versa.

6.1 RS from (weak, allDV+S)-PubMDVS

We first provide a generic construction of ring signature from (weak, allDV+S)-PubMDVS. Roughly, our ring signature scheme based on PubMDVS sets a signer's key sk (in RS) as a verifier's key vsk (in PubMDVS). Then, when generating a signature for a ring R and a message m under a signing key sk , we use the simulation algorithm of PubMDVS by regarding R as a set of designated verifiers' public keys and $\{\text{sk} = \text{vsk}\}$ as a set of designated verifiers' secret keys, respectively. To construct a ring signature from MDVS, public verifiability is essential since ring signature is publicly verifiable by default.

Construction 2. Let $\text{PubMDVS} = (\text{Setup}, \text{SKGen}, \text{VKGen}, \text{Sign}, \text{PubVrfy}, \text{Sim})$ be a PubMDVS scheme. The description of Construction 2 is as follows.

RS.Setup(1^κ): Compute $\text{pp}_{\text{MDVS}} \leftarrow \text{Setup}(1^\kappa)$ and $(\text{spk}, \text{ssk}) \leftarrow \text{SKGen}(\text{pp}_{\text{MDVS}})$. Output $\text{pp}_{\text{RS}} := (\text{pp}_{\text{MDVS}}, \text{spk})$ and discard ssk .

RS.KGen(pp_{RS}): Output $(\text{vpk}, \text{vsk}) \leftarrow \text{VKGen}(\text{pp}_{\text{MDVS}})$.

RS.Sign($\text{pp}_{\text{RS}}, \text{sk}, m, R$): Output $\sigma \leftarrow \text{Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, R, \{\text{sk}\}, m)$.

RS.Verify($\text{pp}_{\text{RS}}, R, m, \sigma$): Output $b \leftarrow \text{PubVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}, R, m, \sigma)$.

We prove that Construction 2 is correct, unforgeable, and anonymous ring signature scheme.

Theorem 16. *If PubMDVS is allDV+S-OTR secure, Construction 2 is correct.*

Proof. Since PubMDVS is allDV+S-OTR secure, due to Theorem 11, RS.Verify outputs 1 with overwhelming probability. $\square$

Theorem 17. *If PubMDVS is allDV+S-OTR secure, Construction 2 is anonymous.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the anonymity of RS. To prove the theorem, we consider the following sequence of games Game_i for $i \in \{0, 1, 2\}$.

Game₀: This is the original anonymity game.

Game₁: In this game, we modify how the challenge is generated. Now, σ_0^* is generated as

$$\sigma_0^* \leftarrow \text{Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}^*, R^*, m^*)$$

instead of $\sigma_0^* \leftarrow \text{Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, R^*, \text{ssk}_0^*, m^*)$.

Game₂: In this game, we further change how the challenge is generated. Now, σ_1^* is generated as

$$\sigma_1^* \leftarrow \text{Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}^*, R^*, m^*)$$

instead of $\sigma_1^* \leftarrow \text{Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, R^*, \text{ssk}_1^*, m^*)$.

We have $\Pr[\text{Game}_2(\kappa) = 1] = 1/2$ since both σ_0^* and σ_1^* have the same distribution in **Game₂**, i.e., independent of the challenge bit b . Therefore, we have

$$\begin{aligned} \text{Adv}_{\text{Const. 2}, \mathcal{A}}^{\text{Anon}}(\kappa) &= \left| \Pr[\text{Game}_0(\kappa) = 1] - \frac{1}{2} \right| \\ &\leq \sum_{i=1}^2 |\Pr[\text{Game}_{i-1}(\kappa) = 1] - \Pr[\text{Game}_i(\kappa) = 1]|. \end{aligned}$$

It remains to show how each $|\Pr[\text{Game}_{i-1}(\kappa) = 1] - \Pr[\text{Game}_i(\kappa) = 1]|$ is upper-bounded. Concretely, we show the following lemmas.

Lemma 1. *Game₀ and Game₁ are indistinguishable assuming PubMDVS is allDV+S-OTR secure. That is, there exists $\mathcal{B}_1$ such that $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| = 2\text{Adv}_{\text{PubMDVS}, \mathcal{B}_1}^{\text{allDV+S-OTR}}(\kappa)$.*

Proof. We show an adversary $\mathcal{B}_1$ that breaks the allDV+S-OTR security of PubMDVS, as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}_1$ queries $\text{O}_{\text{SPK}}()$ and receives spk^* . Then $\mathcal{B}_1$ executes $\mathcal{A}$ on input $\text{pp}_{\text{RS}} := (\text{pp}_{\text{MDVS}}, \text{spk}^*)$.
2. $\mathcal{B}_1$ answers oracle queries from $\mathcal{A}$ as follows.
 - $\text{O}_{\text{VK}}()$: $\mathcal{B}_1$ queries $\text{O}_{\text{VPK}}()$ and receives vpk . $\mathcal{B}_1$ returns vpk to $\mathcal{A}$.
 - $\text{O}_{\text{SK}}(\text{vk})$: $\mathcal{B}_1$ queries $\text{O}_{\text{VSK}}(\text{vk})$ and receives vsk . $\mathcal{B}_1$ returns vsk to $\mathcal{A}$.
 - $\text{O}_{\text{Sig}}(\text{vk}, R, m)$: $\mathcal{B}_1$ queries $\text{O}_{\text{VSK}}(\text{vk})$ and receives vsk . Then, $\mathcal{B}_1$ computes

$$\sigma \leftarrow \text{PubMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, R, \{\text{vsk}\}, m)$$

and returns σ to $\mathcal{A}$.

3. Upon receiving $(\text{vk}_0^*, \text{vk}_1^*, R^*, m^*)$ from $\mathcal{A}$, $\mathcal{B}_1$ queries $\text{O}_{\text{VSK}}(\text{vk}_i^*)$ and receives $\text{vsk}_i^* := \text{sk}_i^*$ for $i \in \{0, 1\}$. Then, it sends $(\text{spk}^*, \mathcal{D}^* := R^*, \mathcal{C}^* := \{\text{sk}_0^*\}, m^*)$ to the challenger to request the challenge and receives σ_0^* . Then, $\mathcal{B}_1$ computes

$$\sigma_1^* \leftarrow \text{PubMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, R^*, \{\text{vsk}_1^*\}, m^*).$$

$\mathcal{B}_1$ flips a coin $d \leftarrow \{0, 1\}$ and gives σ_d^* to $\mathcal{A}$.

4. Finally, $\mathcal{A}$ outputs d' to $\mathcal{B}_1$. $\mathcal{B}_1$ outputs $b' := d \oplus d'$ to its challenger.

We can verify that $\mathcal{B}_1$ perfectly simulates **Game₀** if $b = 0$ and **Game₁** if $b = 1$. In particular, if $\mathcal{A}$'s output $(\text{vk}_0^*, \text{vk}_1^*, R^*, m^*)$ satisfies the condition in line 5 of the anonymity game for RS, then $\mathcal{B}_1$'s output $(\text{spk}^*, \mathcal{D}^* := R^*, \mathcal{C}^* := \{\text{sk}_0^*\}, m^*)$ also satisfies the conditions in line 6 and line 13 of the allDV+S-OTR game for PubMDVS. Also, by definition, $d = d'$ implies $b' = 0$. Therefore, we have

$$\text{Adv}_{\text{PubMDVS}, \mathcal{B}_1}^{\text{allDV+S-OTR}}(\kappa) = \left| \Pr[b = b'] - \frac{1}{2} \right|$$

$$\begin{aligned}
&= \frac{1}{2} |\Pr[b' = 0|b = 0] - \Pr[b' = 0|b = 1]| \\
&= \frac{1}{2} |\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]|.
\end{aligned}$$

Therefore, we have

$$|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| = 2\text{Adv}_{\text{PubMDVS}, \mathcal{B}_1}^{\text{allDV+S-OTR}}(\kappa).$$

□

Lemma 2. *Game₁ and Game₂ are indistinguishable assuming PubMDVS is allDV+S-OTR secure. That is, there exists $\mathcal{B}_2$ such that $|\Pr[\text{Game}_1(\kappa) = 1] - \Pr[\text{Game}_2(\kappa) = 1]| = 2\text{Adv}_{\text{PubMDVS}, \mathcal{B}_2}^{\text{allDV+S-OTR}}(\kappa)$.*

Proof. We show an adversary $\mathcal{B}_2$ that breaks the off-the-record of PubMDVS by using $\mathcal{A}$, as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}_2$ queries $\text{O}_{\text{SPK}}()$ and receives spk^* . Then $\mathcal{B}_2$ executes $\mathcal{A}$ on input $\text{pp}_{\text{RS}} := (\text{pp}_{\text{MDVS}}, \text{spk}^*)$.
2. $\mathcal{B}_2$ answers oracle queries from $\mathcal{A}$ as follows.
 - $\text{O}_{\text{VK}}()$: $\mathcal{B}_2$ queries $\text{O}_{\text{VPK}}()$ and receives vpk . $\mathcal{B}_2$ returns vpk to $\mathcal{A}$.
 - $\text{O}_{\text{SK}}(\text{vk})$: $\mathcal{B}_2$ queries $\text{O}_{\text{VSK}}(\text{vk})$ and receives vsk . $\mathcal{B}_2$ returns vsk to $\mathcal{A}$.
 - $\text{O}_{\text{Sig}}(\text{vk}, \text{R}, \text{m})$: $\mathcal{B}_2$ queries $\text{O}_{\text{VSK}}(\text{vk})$ and receives vsk . Then, $\mathcal{B}_2$ computes

$$\sigma \leftarrow \text{PubMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, \text{R}, \{\text{vsk}\}, \text{m})$$

and returns σ to $\mathcal{A}$.

3. Upon receiving $(\text{vk}_0^*, \text{vk}_1^*, \text{R}^*, \text{m}^*)$ from $\mathcal{A}$, $\mathcal{B}_2$ queries $\text{O}_{\text{Sig}}(\text{spk}^*, \text{R}^*, \text{m}^*)$ and receives σ_0^* . Then, $\mathcal{B}_2$ corrupts sk_1^* via the O_{SK} oracle and outputs $(\text{spk}^*, \mathcal{D}^* := \text{R}^*, \mathcal{C}^* := \{\text{sk}_1^*\}, \text{m}^*)$, $\mathcal{B}_2$ receives σ_1^* as the challenge signature. Then, $\mathcal{B}_2$ flips a coin $d \leftarrow \{0, 1\}$ and gives σ_d^* to $\mathcal{A}$.
4. Finally, $\mathcal{A}$ outputs d' to $\mathcal{B}_2$. $\mathcal{B}_2$ outputs $b' := d \oplus d'$ to its challenger.

Applying the same argument used to transit from Game₀ to Game₁, we have

$$|\Pr[\text{Game}_1(\kappa) = 1] - \Pr[\text{Game}_2(\kappa) = 1]| = 2\text{Adv}_{\text{PubMDVS}, \mathcal{B}}^{\text{allDV+S-OTR}}(\kappa).$$

□

Putting everything together, we have

$$\text{Adv}_{\text{Const. 2}, \mathcal{A}}^{\text{Anon}}(\kappa) = 4\text{Adv}_{\text{PubMDVS}, \mathcal{B}}^{\text{allDV+S-OTR}}(\kappa).$$

Since PubMDVS is allDV+S-OTR secure, $\text{Adv}_{\text{Const. 2}, \mathcal{A}}^{\text{allDV+S-OTR}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 2 is anonymous. □

Theorem 18. *If PubMDVS is allDV+S-OTR and weak-Unf secure, Construction 2 is unforgeable.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the unforgeability of RS. To prove the theorem, we consider the following sequence of games Game_{*i*} for $i \in \{0, 1\}$.

Game₀. This is the original game. By definition, we have $\Pr[\text{Game}_0(\kappa) = 1] = \text{Adv}_{\text{RS}, \mathcal{A}}^{\text{Unf}}(\kappa)$.

Game₁. In this game, we modify how the oracle O_{Sig} generates signatures. When $\text{O}_{\text{Sig}}(\text{vk}, \text{R}, \text{m})$ is queried, O_{Sig} computes signatures as $\sigma \leftarrow \text{Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}, \text{R}, \text{m})$ instead of $\sigma \leftarrow \text{Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \text{R}, \text{sk}, \text{m})$.

To prove the indistinguishability between Game₀ and Game₁, we consider the following sub-hybrids Game_{0,*k*} for $k \in [q_S]$ where $q_S = \text{poly}(\kappa)$ be the number of signing queries made by $\mathcal{A}$.

$\text{Game}_{0,k}$: In this game, the i -th $\text{O}_{\text{Sig}}(\text{vk}, \text{R}, \text{m})$ query for $i \in [k]$ is answered as Game_1 .

We show an adversary $\mathcal{B}$ that breaks the allDV+S-OTR security of PubMDVS by using $\mathcal{A}$. The description of $\mathcal{B}$ is as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}$ samples $k \leftarrow_{\$} [q_S]$ at random. Then, it queries $\text{O}_{\text{SPK}}()$ and receives spk^* . Then $\mathcal{B}$ executes $\mathcal{A}$ on input $\text{pp}_{\text{RS}} := (\text{pp}_{\text{MDVS}}, \text{spk}^*)$.
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ as follows.
 - $\text{O}_{\text{VK}}()$: $\mathcal{B}$ queries $\text{O}_{\text{VPK}}()$ and receives vpk . $\mathcal{B}$ also queries $\text{O}_{\text{VSK}}(\text{vk})$ and receives vsk . $\mathcal{B}$ returns vpk to $\mathcal{A}$.
 - $\text{O}_{\text{SK}}(\text{vk})$: $\mathcal{B}$ returns vsk , which is the corresponding secret key of vk , to $\mathcal{A}$.
 - $\text{O}_{\text{Sig}}(\text{vk}, \text{R}, \text{m})$: For the i -th signing query, if $i < k$, $\mathcal{B}$ queries $\text{O}_{\text{Sig}}(\text{spk}^*, \mathcal{D} := \text{R}, \text{m})$ and receives σ . Else if $i = k$, $\mathcal{B}$ outputs $(\text{spk}^*, \mathcal{D} := \text{R}, \{\text{vk}\}, \text{m})$ to request the challenge and receives σ . Else if $i > k$, $\mathcal{B}$ generates $\sigma \leftarrow \text{PubMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \text{R}, \{\text{sk}\}, \text{m})$ with vk 's secret key sk . $\mathcal{B}$ returns σ to $\mathcal{A}$.
3. Finally, $\mathcal{A}$ outputs $(\text{R}^*, \text{m}^*, \sigma^*)$. $\mathcal{B}$ checks if the output is valid forgery and computes $d \leftarrow \text{RS.Verify}(\text{R}^*, \text{m}^*, \sigma^*)$. $\mathcal{B}$ outputs d to its challenger.

We can verify that $\mathcal{B}$ perfectly simulates $\text{Game}_{0,k-1}$ if $b = 0$ or $\text{Game}_{0,k}$ if $b = 1$. In particular, $\mathcal{B}$'s output $(\text{spk}^*, \mathcal{D} := \text{R}, \{\text{vk}\}, \text{m})$ for the challenge satisfies the condition in line 6 and line 13 of the allDV+S-OTR game due to the if condition in the signing oracle of RS. Therefore, we have

$$\begin{aligned}
\text{Adv}_{\text{PubMDVS}, \mathcal{B}}^{\text{allDV+S-OTR}}(\kappa) &= \left| \Pr[b = b'] - \frac{1}{2} \right| \\
&= \frac{1}{2} |\Pr[b = 0 | b = 0] - \Pr[b = 0 | b = 1]| \\
&= \frac{1}{2} \left| \sum_{i=1}^{q_S} \Pr[b = 0 \wedge i = k | b = 0] - \Pr[b = 0 \wedge i = k | b = 1] \right| \\
&= \frac{1}{2} \left| \sum_{i=1}^{q_S} \Pr[i = k] (\Pr[b = 0 | b = 0 \wedge i = k] - \Pr[b = 0 | b = 1 \wedge i = k]) \right| \\
&= \frac{1}{2q_S} \left| \sum_{i=1}^{q_S} \Pr[b = 0 | b = 0 \wedge i = k] - \Pr[b = 0 | b = 1 \wedge i = k] \right| \\
&= \frac{1}{2q_S} \left| \sum_{i=1}^{q_S} \Pr[\text{Game}_{0,k-1}(\kappa) = 1] - \Pr[\text{Game}_{0,k}(\kappa) = 1] \right| \\
&= \frac{1}{2q_S} |\Pr[\text{Game}_{0,0}(\kappa) = 1] - \Pr[\text{Game}_{0,q_S}(\kappa) = 1]| \\
&= \frac{1}{2q_S} |\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]|.
\end{aligned}$$

We next show that if $\mathcal{A}$ breaks the unforgeability of Construction 2 in Game_1 , there exists an adversary $\mathcal{F}$ that breaks the unforgeability of PubMDVS. The description of $\mathcal{F}$ is as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{F}$ queries $\text{O}_{\text{SPK}}()$ and receives spk^* . Then $\mathcal{F}$ executes $\mathcal{A}$ on input $\text{pp}_{\text{RS}} := (\text{pp}_{\text{MDVS}}, \text{spk}^*)$.
2. $\mathcal{F}$ answers oracle queries from $\mathcal{A}$ as follows.
 - $\text{O}_{\text{VK}}()$: $\mathcal{F}$ queries $\text{O}_{\text{VPK}}()$ to receive vpk and returns vpk to $\mathcal{A}$.

- $O_{SK}(vk)$: $\mathcal{F}$ queries $O_{VSK}(vk)$ to receive vsk and returns vsk to $\mathcal{A}$.
- $O_{Sig}(vk, R, m)$: $\mathcal{F}$ queries $O_{Sig}(spk^*, \mathcal{D} := R, m)$ to receive σ and returns σ to $\mathcal{A}$.

3. Finally, $\mathcal{A}$ outputs (R^*, m^*, σ^*) to $\mathcal{F}$. $\mathcal{F}$ outputs $(spk^*, R^*, m^*, \sigma^*)$ to its challenger.

We can verify that $\mathcal{F}$ perfectly simulates Game_1 and if $\mathcal{A}$'s output (R^*, m^*, σ^*) satisfies the condition in line 4 of the unforgeability game of RS, then $\mathcal{F}$'s output $(spk^*, R^*, m^*, \sigma^*)$ satisfies the condition in line 4 of the unforgeability game of MDVS. Therefore, if $\mathcal{A}$ breaks the unforgeability of Construction 2, $\mathcal{F}$ breaks the unforgeability of PubMDVS. Therefore, we have

$$\Pr[\text{Game}_1(\kappa) = 1] = \text{Adv}_{\text{PubMDVS}, \mathcal{F}}^{\text{weak-Unf}}(\kappa).$$

Putting everything together, we have

$$\text{Adv}_{\text{Const. 2}, \mathcal{A}}^{\text{Unf}}(\kappa) \leq 2q_S \text{Adv}_{\text{PubMDVS}, \mathcal{B}}^{\text{allDV+S-OTR}}(\kappa) + \text{Adv}_{\text{PubMDVS}, \mathcal{F}}^{\text{weak-Unf}}(\kappa).$$

Since PubMDVS is allDV+S-OTR secure and weak-Unf secure, we have $\text{Adv}_{\text{Const. 2}, \mathcal{A}}^{\text{Unf}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 2 is unforgeable. $\square$

6.2 (weak, allDV+S)-PubMDVS from RS

We then provide a generic construction of (weak, allDV+S)-PubMDVS from ring signature. The idea of this construction is interpreting a union set of designated verifiers and a signer as a ring of RS when generating a signature.

Construction 3. Let $\text{RS} = (\text{Setup}, \text{KGen}, \text{Sign}, \text{Verify})$ be a ring signature scheme. The PubMDVS scheme Construction 3 based on RS is as follows.

$\text{Setup}(1^\kappa)$: Output $pp \leftarrow \text{RS.Setup}(1^\kappa)$.

$\text{SKGen}(pp)$: Output $(spk, ssk) \leftarrow \text{RS.KGen}(pp)$.

$\text{VKGen}(pp)$: Output $(vpk, vsk) \leftarrow \text{RS.KGen}(pp)$.

$\text{Sign}(pp, ssk, \mathcal{D}, m)$: Output $\sigma \leftarrow \text{RS.Sign}(pp, ssk, \mathcal{D} \cup \{spk\}, m)$.

$\text{PubVrfy}(pp, spk, \mathcal{D}, m, \sigma)$: Output $b \leftarrow \text{RS.Verify}(pp, \mathcal{D} \cup \{spk\}, m, \sigma)$.

$\text{Sim}(pp, spk, \mathcal{D}, \mathcal{C}, m)$: Pick a secret key $vsk \leftarrow \mathcal{C}^8$ and output $\sigma \leftarrow \text{RS.Sign}(pp, vsk, \mathcal{D} \cup \{spk\}, m)$.

We show that Construction 3 is a secure (weak, allDV+S)-PubMDVS scheme. More precisely, we prove that Construction 3 is correct, weak-Unf secure, and allDV+S-OTR secure.

Theorem 19. *If RS is correct, Construction 3 is correct.*

Proof. The correctness of RS directly implies the correctness of Construction 3. $\square$

Theorem 20. *If RS is anonymous, Construction 3 is allDV+S-OTR secure.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the allDV+S-OTR of Construction 3. We show an adversary $\mathcal{B}$ that breaks the anonymity of RS by using $\mathcal{A}$, as follows.

1. Upon receiving pp_{RS} , $\mathcal{B}$ executes $\mathcal{A}$ on input $pp_{MDVS} := pp_{RS}$.
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ as follows.

⁸The choice of vsk is arbitrary. We can choose it randomly or the smallest/largest key in a lexicographical order.

- $O_{\text{SPK}}()$: $\mathcal{B}$ queries $O_{\text{VK}}()$ to receive vk and returns $\text{spk} := \text{vk}$ to $\mathcal{A}$.
 - $O_{\text{SSK}}(\text{spk})$: $\mathcal{B}$ queries $O_{\text{SK}}(\text{spk})$ to receive sk and returns $\text{ssk} := \text{sk}$ to $\mathcal{A}$.
 - $O_{\text{VPK}}()$: $\mathcal{B}$ queries $O_{\text{VK}}()$ to receive vk and returns $\text{vpk} := \text{vk}$ to $\mathcal{A}$.
 - $O_{\text{VSK}}(\text{vpk})$: $\mathcal{B}$ queries $O_{\text{SK}}()$ to receive sk and returns $\text{vsk} := \text{sk}$ to $\mathcal{A}$.
 - $O_{\text{Sig}}(\text{spk}, \mathcal{D}, \text{m})$: $\mathcal{B}$ queries $O_{\text{Sig}}(\text{spk}, \text{R} := \mathcal{D} \cup \{\text{spk}\}, \text{m})$ to receive σ and returns σ to $\mathcal{A}$.
3. Upon receiving $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \text{m}^*)$, $\mathcal{B}$ checks if $\mathcal{A}$ does not trivialize the game. Then, $\mathcal{B}$ picks a key $(\text{vpk}^*, \text{vsk}^*)$ such that $\text{vsk}^* \in \mathcal{C}^*$ and outputs $(\text{spk}^*, \text{vpk}^*, \text{R}^* := \mathcal{D}^* \cup \{\text{spk}^*\}, \text{m}^*)$ to its challenger and receives the challenge σ^* . $\mathcal{B}$ returns σ^* to $\mathcal{A}$.
 4. Finally, $\mathcal{A}$ outputs a bit b' . $\mathcal{B}$ outputs b' to its challenger.

We can verify that $\mathcal{B}$ perfectly simulates the allDV+S-OTR game against $\mathcal{A}$. Moreover, if the condition in line 6 and line 13 of the OTR game holds, then the condition in line 5 of the anonymity game also holds. Therefore, if $\mathcal{A}$ breaks the allDV+S-OTR of Construction 3, $\mathcal{B}$ breaks the anonymity of RS, i.e.,

$$\text{Adv}_{\text{Const. 3}, \mathcal{A}}^{\text{allDV+S-OTR}}(\kappa) = \text{Adv}_{\text{RS}, \mathcal{B}}^{\text{Anon}}(\kappa).$$

Since RS is anonymous, we have $\text{Adv}_{\text{Const. 3}, \mathcal{A}}^{\text{allDV+S-OTR}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 3 is allDV+S-OTR secure. $\square$

Theorem 21. *If RS is unforgeable, Construction 3 is weak-Unf secure.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the weak-Unf of Construction 3. We show an adversary $\mathcal{B}$ that breaks the unforgeability of RS by using $\mathcal{A}$, as follows.

1. Upon receiving pp_{RS} , $\mathcal{B}$ executes $\mathcal{A}$ on input $\text{pp}_{\text{MDVS}} := \text{pp}_{\text{RS}}$.
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ as follows.
 - $O_{\text{SPK}}()$: $\mathcal{B}$ queries $O_{\text{VK}}()$ and receives vk . $\mathcal{B}$ returns $\text{spk} := \text{vk}$ to $\mathcal{A}$.
 - $O_{\text{SSK}}(\text{spk})$: $\mathcal{B}$ queries $O_{\text{SK}}(\text{spk})$ and receives sk . $\mathcal{B}$ returns $\text{ssk} := \text{sk}$ to $\mathcal{A}$.
 - $O_{\text{VPK}}()$: $\mathcal{B}$ queries $O_{\text{VK}}()$ and receives vk . $\mathcal{B}$ returns $\text{spk} := \text{vk}$ to $\mathcal{A}$.
 - $O_{\text{VSK}}(\text{vpk})$: $\mathcal{B}$ queries $O_{\text{SK}}(\text{vpk})$ and receives sk . $\mathcal{B}$ returns $\text{ssk} := \text{sk}$ to $\mathcal{A}$.
 - $O_{\text{Sig}}(\text{spk}, \mathcal{D}, \text{m})$: $\mathcal{B}$ queries $O_{\text{Sig}}(\text{spk}, \text{R} := \mathcal{D} \cup \{\text{spk}\}, \text{m})$ and receives σ . It returns σ to $\mathcal{A}$.
3. Finally, $\mathcal{A}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^*)$ as forgery. $\mathcal{B}$ outputs $(\text{R}^* := \mathcal{D}^* \cup \{\text{spk}^*\}, \text{m}^*, \sigma^*)$ to its challenger as its forgery.

We can verify that $\mathcal{B}$ perfectly simulates the weak-Unf game against $\mathcal{A}$. Moreover, if the condition in line 4 of the weak-Unf game holds, then the condition in line 5 of the unforgeability game of RS holds. Therefore, if $\mathcal{A}$ breaks the weak-Unf security of Construction 3, $\mathcal{B}$ breaks the unforgeability of RS, i.e.,

$$\text{Adv}_{\text{Const. 3}, \mathcal{A}}^{\text{weak-Unf}}(\kappa) = \text{Adv}_{\text{RS}, \mathcal{B}}^{\text{Unf}}(\kappa).$$

Since RS is unforgeable, we have $\text{Adv}_{\text{Const. 3}, \mathcal{A}}^{\text{weak-Unf}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 3 is weak-Unf secure. $\square$

7 weak-PubMDVS Implies weak-PrivMDVS

In this section, we show that (weak, yyy)-PubMDVS implies (weak, yyy)-PrivMDVS for $yyy \in \{\text{simDV}, \text{simDV}+\text{S}, \text{allDV}+\text{S}\}$.

Construction 4. Let PubMDVS be a PubMDVS scheme. The PrivMDVS scheme Construction 4 is as follows.

Setup(1^κ): Output $\text{pp} \leftarrow \text{PubMDVS.Setup}(1^\kappa)$.

SKGen(pp): Output $(\text{spk}, \text{ssk}) \leftarrow \text{PubMDVS.SKGen}(\text{pp})$.

VKGen(pp): Output $(\text{vpk}, \text{vsk}) \leftarrow \text{PubMDVS.VKGen}(\text{pp})$.

Sign(pp, ssk, $\mathcal{D}$, m): Output $\sigma \leftarrow \text{PubMDVS.Sign}(\text{pp}, \text{ssk}, \mathcal{D}, m)$.

PrivVrfy(pp, spk, vsk, $\mathcal{D}$, m, σ) : Output $b \leftarrow \text{PubMDVS.PubVrfy}(\text{pp}, \text{spk}, \mathcal{D}, m, \sigma)$.

Sim(pp, spk, $\mathcal{D}$, $\mathcal{C}$, m): Output $\sigma \leftarrow \text{PubMDVS.Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, m)$.

We can see that each algorithm of Construction 4 just runs the corresponding algorithm of PubMDVS with the same inputs except that the verification algorithm ignores the verification secret key.

We show that Construction 4 satisfies correctness, consistency, weak-Unf, and yyy-OTR security if PubMDVS satisfies the same security properties. Formally, we prove the following theorems.

Theorem 22. *If PubMDVS is correct, Construction 4 is correct.*

Proof. The correctness of Construction 4 is immediately followed by the correctness of PubMDVS. $\square$

Theorem 23. *Construction 4 has consistency.*

Proof. The consistency of Construction 4 is immediately followed by the fact that PubMDVS has consistency by default and PrivMDVS.PrivVrfy runs PubMDVS.PubVrfy internally, ignoring the input vsk. $\square$

Theorem 24. *If PubMDVS is yyy-OTR secure, Construction 4 is yyy-OTR secure.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the yyy-OTR security of Construction 4. We show an adversary $\mathcal{B}$ that breaks the yyy-OTR security of PubMDVS by using $\mathcal{A}$, as follows.

1. Upon receiving pp , $\mathcal{B}$ executes $\mathcal{A}$ on input pp .
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ by using the corresponding oracle provided to $\mathcal{B}$. Note that $\mathcal{B}$ uses PubMDVS.PubVrfy algorithm to answer O_{Ver} queries.
3. Upon receiving $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, m^*)$, $\mathcal{B}$ checks the if statements (cf. line 6 of yyy-OTR game). Then, $\mathcal{B}$ outputs $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, m^*)$ to its challenger and receives the challenge σ^* . $\mathcal{B}$ returns σ^* to $\mathcal{A}$.
4. Finally, $\mathcal{A}$ outputs a bit b' . $\mathcal{B}$ outputs b' to its challenger.

We can verify that $\mathcal{B}$ perfectly simulates the yyy-OTR game against $\mathcal{A}$. Moreover, if $\mathcal{A}$'s activities satisfy the conditions in line 6 and line 13 of the yyy-OTR game, $\mathcal{B}$ so does. Therefore, if $\mathcal{A}$ breaks the yyy-OTR security of Construction 4, $\mathcal{B}$ breaks the yyy-OTR security of PubMDVS, i.e.,

$$\text{Adv}_{\text{Const. 4}, \mathcal{A}}^{\text{yyy-OTR}}(\kappa) = \text{Adv}_{\text{PubMDVS}, \mathcal{D}}^{\text{yyy-OTR}}(\kappa).$$

Since PubMDVS is yyy-OTR secure, we have $\text{Adv}_{\text{Const. 4}, \mathcal{A}}^{\text{yyy-OTR}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 4 is yyy-OTR secure. $\square$

Theorem 25. *If PubMDVS is weak-Unf secure, Construction 4 is weak-Unf secure.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the weak-Unf security of Construction 4. We show an adversary $\mathcal{B}$ that breaks the weak-Unf security of PubMDVS by using $\mathcal{A}$, as follows.

1. Upon receiving pp , $\mathcal{B}$ executes $\mathcal{A}$ on input pp .
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ by using the corresponding oracle. Note that $\mathcal{B}$ uses PubMDVS.PubVrfy algorithm to answer O_{Ver} queries.
3. Finally, $\mathcal{A}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^*)$ as forgery. $\mathcal{B}$ outputs it to its challenger as its forgery.

We can verify that $\mathcal{B}$ perfectly simulates the weak-Unf game against $\mathcal{A}$. In addition, if $\mathcal{A}$ breaks the weak-Unf security of Construction 4, $\mathcal{B}$ also breaks the weak-Unf security of PubMDVS, i.e.,

$$\text{Adv}_{\text{Const. 4}, \mathcal{A}}^{\text{weak-Unf}}(\kappa) = \text{Adv}_{\text{PubMDVS}, \mathcal{B}}^{\text{weak-Unf}}(\kappa).$$

Since PubMDVS is weak-Unf secure, we have $\text{Adv}_{\text{Const. 4}, \mathcal{A}}^{\text{weak-Unf}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 4 is weak-Unf secure. $\square$

8 (strong, simDV+S)-PrivMDVS based on (weak, allDV+S)-PrivMDVS

We show a generic construction of (strong, simDV+S)-PrivMDVS (without consistency) from (weak, allDV+S)-PrivMDVS plus IND-CCA secure PKE and strong OTS.

Construction 5. Let PrivMDVS be a PrivMDVS scheme with the signature length ℓ , PKE a PKE scheme, and OTS an OTS scheme. The PrivMDVS scheme (Construction 5) is as follows.

Setup(1^κ): Compute $\text{pp}_{\text{MDVS}} \leftarrow \text{PrivMDVS.Setup}(1^\kappa)$ and $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\kappa)$. Output $\text{pp} := (\text{pp}_{\text{MDVS}}, \text{pp}_{\text{PKE}})$.

SKGen(pp): Output $(\text{spk}, \text{ssk}) \leftarrow \text{PrivMDVS.SKGen}(\text{pp}_{\text{MDVS}})$.

VKGen(pp): Compute $(\hat{\text{vpk}}, \hat{\text{vsk}}) \leftarrow \text{PrivMDVS.VKGen}(\text{pp}_{\text{MDVS}})$ and $(\text{pk}, \text{sk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$. Output $(\text{vpk}, \text{vsk}) := ((\hat{\text{vpk}}, \text{pk}), (\hat{\text{vsk}}, \text{sk}))$

Sign($\text{pp}, \text{ssk}, \mathcal{D}, \text{m}$): Let $\mu := |\mathcal{D}|$ and $\{(\hat{\text{vpk}}_i, \text{pk}_i)\}_{i \in [\mu]} := \mathcal{D}^9$. Firstly, generate $(\text{ovk}, \text{osk}) \leftarrow \text{OTS.KeyGen}(1^\kappa)$. Then, for each $i \in [\mu]$, compute

$$\begin{aligned} \hat{\sigma}_i &\leftarrow \text{PrivMDVS.Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}, \{\hat{\text{vpk}}_i\}, \text{m} \parallel \mathcal{D}), \\ C_i &\leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_i, \hat{\sigma}_i \parallel \text{ovk}). \end{aligned}$$

Then, set $\Sigma := (C_i)_{i \in [\mu]}$ and compute $\text{osig} \leftarrow \text{OTS.Sign}(\text{osk}, \text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma)$. Finally, output the signature $\sigma := (\Sigma, \text{ovk}, \text{osig})$.

PrivVrfy($\text{pp}, \text{spk}, \text{vsk}, \mathcal{D}, \text{m}, \sigma$) : Parse $(\Sigma, \text{ovk}, \text{osig}) := \sigma$ and $(\hat{\text{vsk}}, \text{sk}) := \text{vsk}$. Firstly, if $\text{vpk} \notin \mathcal{D}$ or $\text{OTS.Verify}(\text{ovk}, \text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma, \text{osig}) = 0$, output 0. Else, extract the i -th entry C_i from Σ^{10} and compute $\hat{\sigma} \parallel \text{ovk}' \leftarrow \text{PKE.Dec}(\text{pp}_{\text{PKE}}, \text{sk}, C_i)$. If $\hat{\sigma} = 0^\ell$ or $\text{ovk}' \neq \text{ovk}$, output 0. Otherwise, output $b \leftarrow \text{PrivMDVS.PrivVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}, \hat{\text{vsk}}, \{\hat{\text{vpk}}\}, \text{m} \parallel \mathcal{D}, \hat{\sigma})$.

Sim($\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m}$): Let $\mu := |\mathcal{D}|$ and $\{(\hat{\text{vpk}}_i, \text{pk}_i)\}_{i \in [\mu]} := \mathcal{D}$. Firstly, generate $(\text{ovk}, \text{osk}) \leftarrow \text{OTS.KeyGen}(1^\kappa)$. For each $i \in [\mu]$, if the secret key $\text{vsk}_i = (\hat{\text{vsk}}, \text{sk}_i)$ corresponding to vpk_i is in $\mathcal{C}$, compute

$$\hat{\sigma}_i \leftarrow \text{PrivMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \{\hat{\text{vpk}}_i\}, \{\hat{\text{vsk}}_i\}, \text{m} \parallel \mathcal{D}),$$

⁹We assume $\mathcal{D}$ is sorted in a lexicographical order.

¹⁰We assume vpk is the i -th entry in $\mathcal{D}$.

$$C_i \leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_i, \hat{\sigma}_i \parallel \text{ovk}).$$

Else, compute

$$C_i \leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_i, 0^\ell \parallel \text{ovk}).$$

Then, set $\Sigma := (C_i)_{i \in [\mu]}$ and compute $\text{osig} \leftarrow \text{OTS.Sign}(\text{osk}, \text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma)$. Finally, output the signature $\sigma := (\Sigma, \text{ovk}, \text{osig})$.

Remark 26. The signature size of Construction 5 is proportional to the number of the designated verifiers $|\mathcal{D}|$. This is inherent when subset simulation and strong unforgeability are considered [DHM⁺20, Theorem 1].

We prove that Construction 5 satisfies correctness, **strong-Unf** security, and **simDV+S-OTR** security. Note that it does not have consistency, but it can be added by using our generic conversion proposed in Section 9.

Theorem 27. *If PrivMDVS, OTS, and PKE are correct, Construction 5 is correct.*

Proof. The correctness of Construction 5 is followed by the correctness of PrivMDVS, OTS, and PKE. $\square$

Theorem 28. *Let Q_V be the maximum number of O_{VPK} queries made by an adversary. If PrivMDVS is allDV+S-OTR secure, OTS is sEUF-CMA secure, and PKE is Q_V -IND-CCA-C secure, Construction 5 is simDV+S-OTR secure.*

Proof. Let $\mathcal{A}$ be an adversary that breaks the simDV+S-OTR security of Construction 5. To prove the theorem, we consider the following sequence of games from Game_0 to Game_4 . In the following, let $\sigma^* = (\Sigma^*, \text{ovk}^*, \text{osig}^*)$ be the challenge signature and $(\text{spk}^*, \mathcal{D}^*, \text{m}^*)$ be the data used to generate σ^* . Also, we assume the game keeps the list $L^* = \{(C_i^*, \text{pk}_i)\}_{i \in [|\mathcal{D}^*|]}$ that stores pairs of the PKE ciphertext in the challenge signature Σ^* and the designated verifier's PKE public key used to generate C_i^* .

Game₀: This is the original allDV+S-OTR game.

Game₁: In this game, $\text{O}_{\text{Ver}}(\text{spk}, \text{vpk}, \mathcal{D}, \text{m}, \sigma)$ immediately outputs 0 if the event **Forge** occurs.

Forge: The input of the oracle satisfies $\text{ovk} = \text{ovk}^*$, $(\text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma, \text{osig}) \neq (\text{spk}^* \parallel \mathcal{D}^* \parallel \text{m}^* \parallel \Sigma^*, \text{osig}^*)$, and $\text{OTS.Verify}(\text{ovk}, \text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma, \text{osig}) = 1$.

Due to this modification, when $\text{ovk} = \text{ovk}^*$, O_{Ver} may return 1 only when $(\text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma, \text{osig}) = (\text{spk}^* \parallel \mathcal{D}^* \parallel \text{m}^* \parallel \Sigma^*, \text{osig}^*)$. However, in this case, $\mathcal{A}$ cannot send any $\text{vpk} \in \mathcal{D}^* \setminus \mathcal{C}_{\text{vpk}}^*$ to O_{Ver} oracle along with them due to the winning condition in Line 13. Thus, we can assume when $(\text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma, \text{osig}) = (\text{spk}^* \parallel \mathcal{D}^* \parallel \text{m}^* \parallel \Sigma^*, \text{ovk}^*, \text{osig}^*)$, the oracle input vpk satisfies $\text{vpk} \in \mathcal{C}_{\text{vpk}}^*$ (and the game/reduction knows the secret key of such a verifier's public key).

Game₂: In this game, $\text{O}_{\text{Ver}}(\text{spk}, \text{vpk}, \mathcal{D}, \text{m}, \sigma)$ immediately outputs 0 if the extracted PKE ciphertext C_i satisfies $(C_i, \text{pk}) \in L^*$ and $\text{ovk} \neq \text{ovk}^*$, where $\text{pk} \in \text{vpk}$.

Due to this modification, when $\text{ovk} \neq \text{ovk}^*$, O_{Ver} may return 1 only when $(C, \text{pk}) \notin L^*$.

Game₃: In this game, we modify how the challenge signature is generated. Now, each individual signature $\hat{\sigma}_i$ for $\text{vpk}_i^* \in \mathcal{D}^*$ contained in σ_0^* is generated as $\hat{\sigma}_i \leftarrow \text{PrivMDVS.Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}^*, \{\text{vpk}_i^*\}, \text{m}^* \parallel \mathcal{D}^*)$ instead of $\hat{\sigma}_i \leftarrow \text{PrivMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, \{\text{vpk}_i^*\}, \{\text{vsk}_i^*\}, \text{m}^* \parallel \mathcal{D}^*)$.

Game₄: In this game, we further modify how the challenge signature is generated. Now, each individual ciphertext C_i in σ_0^* is generated as $C_i \leftarrow \text{PKE.Enc}(\text{pp}_{\text{PKE}}, \text{pk}_i, \hat{\sigma}_i \parallel \text{ovk})$. That is, all ciphertexts contain the corresponding signature $\hat{\sigma}_i$ (generated by PrivMDVS.Sign) and no ciphertext encrypts 0.

By definition, we have $|\Pr[\text{Game}_0(\kappa) = 1] - 1/2| = \text{Adv}_{\text{Const. 5}, \mathcal{A}}^{\text{simDV+S-OTR}}(\kappa)$. Then, we can estimate the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{Const. 5}, \mathcal{A}}^{\text{simDV+S-OTR}}(\kappa) \leq \sum_{i=1}^4 |\Pr[\text{Game}_{i-1}(\kappa) = 1] - \Pr[\text{Game}_i(\kappa) = 1]| + \left| \Pr[\text{Game}_4(\kappa) = 1] - \frac{1}{2} \right|.$$

It remains to show how each $|\Pr[\text{Game}_{i-1}(\kappa) = 1] - \Pr[\text{Game}_i(\kappa) = 1]|$ and $|\Pr[\text{Game}_4(\kappa) = 1] - 1/2|$ is upper-bounded. Concretely, we can show the following lemmas.

Lemma 3. *Game₀ and Game₁ are indistinguishable assuming OTS is sEUF-CMA secure. That is, there exists an adversary $\mathcal{B}_1$ such that $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \text{Adv}_{\text{OTS}, \mathcal{B}_1}^{\text{sEUF-CMA}}(\kappa)$.*

Proof. Unless Forge occurs, Game₀ and Game₁ are identical. Thus, we have $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \Pr[\text{Forge}]$. We show that the probability of occurring Forge is negligible if OTS is sEUF-CMA secure.

We show an adversary $\mathcal{B}_1$ that breaks the sEUF-CMA security of OTS by using $\mathcal{A}$ that causes Forge. The description of $\mathcal{B}_1$ is as follows.

1. Upon receiving the challenge verification key ovk^* , $\mathcal{B}_1$ setups the public parameter pp of Construction 5 and executes $\mathcal{A}$ on input pp .
2. $\mathcal{B}_1$ answers oracle queries from $\mathcal{A}$ as in the previous game except that O_{Ver} answers as follows.
 - $\text{O}_{\text{Ver}}(\text{spk}, \text{vpk}, \mathcal{D}, \text{m})$: $\mathcal{B}$ checks if Forge occurs for the received $\sigma = (\Sigma, \text{ovk}, \text{osig})$. If so, $\mathcal{B}_1$ outputs $(\text{ovk}, \text{spk} \parallel \mathcal{D} \parallel \text{m} \parallel \Sigma, \text{osig})$ as a forged signature. Otherwise, $\mathcal{B}$ proceeds identically to the previous game.
3. Upon receiving $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \text{m}^*)$, $\mathcal{B}_1$ creates the challenge signature with the challenge OTS vk ovk^* . That is, $\mathcal{B}_1$ sends $(\text{ovk}^*, \text{spk}^* \parallel \mathcal{D}^* \parallel \text{m}^* \parallel \Sigma^*)$ and receives osig^* . Other procedures are identical to the previous game.
4. Finally, $\mathcal{A}$ outputs a bit b' .

We can verify that $\mathcal{B}_1$ perfectly simulates Game₁. We assume Forge has occurred during the game. Then, $\mathcal{B}_1$ breaks the sEUF-CMA security of OTS when Forge occurs. Therefore, we have $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \Pr[\text{Forge}] \leq \text{Adv}_{\text{OTS}, \mathcal{B}_1}^{\text{sEUF-CMA}}(\kappa)$. $\square$

Lemma 4. *Game₁ and Game₂ are identical, i.e., $\Pr[\text{Game}_1(\kappa) = 1] = \Pr[\text{Game}_2(\kappa) = 1]$.*

Proof. When the condition is satisfied, in Game₁, the O_{Ver} oracle correctly decrypts $C \in \Sigma^*$ with sk corresponding to $\text{pk} \in \text{vpk}$. Due to the correctness of PKE, the decrypted plaintext includes ovk^* by construction. Also, the O_{Ver} oracle checks if the attacked OTS verification key is identical to the encrypted one. Therefore, when the condition is satisfied, the O_{Ver} oracle in Game₁ outputs 0. Thus, Game₁ and Game₂ are identical. $\square$

Lemma 5. *Game₂ and Game₃ are indistinguishable assuming PrivMDVS is allDV+S-OTR secure. That is, there exists an adversary $\mathcal{B}_2$ such that $|\Pr[\text{Game}_2(\kappa) = 1] - \Pr[\text{Game}_3(\kappa) = 1]| = 2|\mathcal{D}^*| \text{Adv}_{\text{PrivMDVS}, \mathcal{B}_2}^{\text{allDV+S-OTR}}(\kappa)$.*

Proof. To prove the indistinguishability between Game₂ and Game₃, we consider the following sub-hybrids Game_{2,k} for $k \in [\nu]$ where $\nu = |\mathcal{D}^*|$ is the number of the designated verifiers in $\mathcal{D}^*$:

Game_{2,k}. In this game, the signatures $\hat{\sigma}_i$ for the 1st to k -th designated verifiers are generated as in Game₃.

We show an adversary $\mathcal{B}_2$ that breaks the allDV+S-OTR security of PrivMDVS by using $\mathcal{A}$. The description of $\mathcal{B}_2$ is as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}_2$ samples $k \leftarrow [\nu]$ and generates pp_{PKE} by itself. Then $\mathcal{B}_2$ executes $\mathcal{A}$ on input $\text{pp} := (\text{pp}_{\text{MDVS}}, \text{pp}_{\text{PKE}})$.

2. $\mathcal{B}_2$ answers oracle queries from $\mathcal{A}$ as follows.

- $\text{O}_{\text{SPK}}()$, $\text{O}_{\text{SSK}}(\text{spk})$, $\text{O}_{\text{VPK}}()$, $\text{O}_{\text{VSK}}(\text{vpk})$: $\mathcal{B}$ answers these oracle queries by using its corresponding oracles. Note that $\mathcal{B}_2$ generate PKE key pairs by itself, thus knows the PKE decryption keys.
- $\text{O}_{\text{Sig}}(\text{spk}, \mathcal{D}, m)$: $\mathcal{B}$ generates the signature as $\text{PrivMDVS}'.Sign except that $\hat{\sigma}_i$ is generated by querying $(\text{spk}, \{\hat{\text{vpk}}_i\}, m \parallel \mathcal{D})$ to its O_{Sig} oracle.$
- $\text{O}_{\text{Ver}}(\text{spk}, \text{vpk}, \mathcal{D}, m)$: $\mathcal{B}$ verifies the signature as in the previous game except that $\hat{\sigma}_i$ is verified by querying $(\text{spk}, \hat{\text{vpk}}_i, \{\hat{\text{vpk}}_i\}, m \parallel \mathcal{D}, \hat{\sigma}_i)$ to its O_{Ver} oracle.

3. Upon receiving $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, m^*)$, $\mathcal{B}_2$ performs as in the previous game, except that it computes the k -th signature $\hat{\sigma}_k$ in σ_0^* by using its challenge request, i.e., corrupt vsk_k via O_{VSK} , outputs $(\text{spk}^*, \{\text{vpk}_k\}, \{\text{vsk}_k\}, m^* \parallel \mathcal{D}^*)$ to its challenger and receives $\hat{\sigma}_k$. Then, $\mathcal{B}_2$ flips a coin d and returns σ_d^* to $\mathcal{A}$.

4. Finally, $\mathcal{A}$ outputs a bit d' . $\mathcal{B}_2$ outputs $b' := d \oplus d'$ to its challenger.

We can verify that $\mathcal{B}_2$ perfectly simulates $\text{Game}_{2,k-1}$ if $b = 0$ and $\text{Game}_{2,k}$ if $b = 1$. Also, by definition, $d = d'$ if and only if $b' = 0$. Therefore, we have

$$\begin{aligned}
\text{Adv}_{\text{PrivMDVS}, \mathcal{B}_2}^{\text{allDV+S-OTR}}(\kappa) &= \left| \Pr[b = b'] - \frac{1}{2} \right| \\
&= \frac{1}{2} |\Pr[b' = 0|b = 0] - \Pr[b' = 0|b = 1]| \\
&= \frac{1}{2} \left| \sum_{i=1}^{\mu} \Pr[b' = 0 \wedge i = k|b = 0] - \Pr[b' = 0 \wedge i = k|b = 1] \right| \\
&= \frac{1}{2\mu} \left| \sum_{i=1}^{\mu} \Pr[b' = 0|b = 0 \wedge i = k] - \Pr[b' = 0|b = 1 \wedge i = k] \right| \\
&= \frac{1}{2\mu} \left| \sum_{i=1}^{\mu} \Pr[d = d'|b = 0 \wedge i = k] - \Pr[d = d'|b = 1 \wedge i = k] \right| \\
&= \frac{1}{2\mu} \left| \sum_{i=1}^{\mu} \Pr[\text{Game}_{2,k-1}(\kappa) = 1] - \Pr[\text{Game}_{2,k}(\kappa) = 1] \right| \\
&= \frac{1}{2\mu} |\Pr[\text{Game}_{2,0}(\kappa) = 1] - \Pr[\text{Game}_{2,\mu}(\kappa) = 1]| \\
&= \frac{1}{2\mu} |\Pr[\text{Game}_2(\kappa) = 1] - \Pr[\text{Game}_3(\kappa) = 1]|.
\end{aligned}$$

Therefore, we have $|\Pr[\text{Game}_2(\kappa) = 1] - \Pr[\text{Game}_3(\kappa) = 1]| = 2\mu \text{Adv}_{\text{PrivMDVS}, \mathcal{B}_2}^{\text{allDV+S-OTR}}(\kappa)$. $\square$

Lemma 6. *Game₃ and Game₄ are indistinguishable assuming PKE is Q_V -IND-CCA-C secure, where Q_V is the maximum number of O_{VPK} queries made by $\mathcal{A}$. That is, there exists an adversary $\mathcal{B}_3$ such that $|\Pr[\text{Game}_3(\kappa) = 1] - \Pr[\text{Game}_4(\kappa) = 1]| \leq 2\text{Adv}_{\text{PKE}, \mathcal{B}_3}^{Q_V\text{-IND-CCA-C}}(\kappa)$.*

Proof. To prove the indistinguishability between Game_3 and Game_4 , we show an adversary $\mathcal{B}_3$ that breaks the Q_V -IND-CCA-C security of PKE by using $\mathcal{A}$. The description of $\mathcal{B}_3$ is as follows.

1. Upon receiving pp_{PKE} and public keys $\{\text{vpk}_i\}_{i \in [q_{\text{DV}}]}$, $\mathcal{B}_3$ generates pp_{MDVS} and executes $\mathcal{A}$ on input $\text{pp} := (\text{pp}_{\text{MDVS}}, \text{pp}_{\text{PKE}})$.
2. $\mathcal{B}_3$ answers oracle queries from $\mathcal{A}$ as follows.
 - $\text{O}_{\text{SPK}}()$: $\mathcal{B}_3$ generates (spk, ssk) by itself and returns spk .

- $O_{SSK}(spk)$: $\mathcal{B}_3$ returns vsk corresponding to vpk .
 - $O_{VPK}()$: Let this be the I -th signing query. $\mathcal{B}_3$ uses the I -th PKE public key pk_I as the I -th verifier's key. $\mathcal{B}_3$ generates (vpk, vsk) by itself and returns $vpk := (\hat{vpk}, pk_I)$.
 - $O_{VSK}(vpk)$: $\mathcal{B}_3$ queries $O_{Corr}(pk)$ and receives sk . $\mathcal{B}_3$ returns $(\hat{vsk}, sk)$ to $\mathcal{A}$, where $\hat{vsk}$ is the secret key corresponding to $\hat{vpk}$.
 - $O_{Sig}(spk, \mathcal{D}, m)$: $\mathcal{B}$ generates the signature as $PrivMDVS'.Sign$.
 - $O_{Ver}(spk, vpk, \mathcal{D}, m)$: $\mathcal{B}_3$ verifies the signature as in the previous game except that C_i is decrypted by sending (pk, C_i) to its decryption oracle if pk is not corrupted (if it is corrupted, $\mathcal{B}_3$ knows the secret key sk and thus can decrypt C_i by itself). Note that due to the changes made in $Game_1$ and $Game_2$, any PKE ciphertext can be sent to the decryption oracle.
3. Upon receiving $(spk^*, \mathcal{D}^*, \mathcal{C}^*, m^*)$ from $\mathcal{A}$, $\mathcal{B}_3$ computes the challenge σ_0^* as in the previous game except that the ciphertexts for the uncorrupted verifiers $vpk_i \in \mathcal{D}^* \setminus \mathcal{C}_{vpk}$ are generated by its challenge oracle. That is, $\mathcal{B}_3$ queries $(vpk_i, 0 || ovk, \sigma_i || ovk)$ for such vpk_i and receives C_i . Then, $\mathcal{B}_3$ flips a coin d and returns σ_d^* to $\mathcal{A}$.
 4. Finally, $\mathcal{A}$ outputs a bit d' . $\mathcal{B}_3$ outputs $b' := d \oplus d'$ to its challenger.

We can verify that $\mathcal{B}_3$ perfectly simulates $Game_3$ if $b = 0$ and $Game_4$ if $b = 1$. (Note that $\mathcal{A}$ is restricted not to corrupt $vpk_i \in \mathcal{D}^* \setminus \mathcal{C}_{vpk}$ after it requests the challenge.) Also, by definition, $d = d'$ implies $b' = 1$. Therefore, we have

$$Adv_{PKE, \mathcal{B}_3}^{Q_V-IND-CCA-C}(\kappa) = \left| \Pr[b = b'] - \frac{1}{2} \right| = \frac{1}{2} |\Pr[Game_3(\kappa) = 1] - \Pr[Game_4(\kappa) = 1]|.$$

We obtain $|\Pr[Game_3(\kappa) = 1] - \Pr[Game_4(\kappa) = 1]| = 2Adv_{PKE, \mathcal{B}_3}^{Q_V-IND-CCA-C}(\kappa)$. $\square$

Putting everything together, we have

$$Adv_{Const. 5, \mathcal{A}}^{simDV+S-OTR}(\kappa) \leq Adv_{OTS, \mathcal{B}_1}^{sEUFCMA}(\kappa) + 2|\mathcal{D}^*| Adv_{PrivMDVS, \mathcal{B}_2}^{allDV+S-OTR}(\kappa) + 2Adv_{PKE, \mathcal{B}_3}^{Q_V-IND-CCA-C}(\kappa).$$

Since OTS is sEUFCMA secure, PrivMDVS is allDV+S-OTR secure, and PKE is $Q_V-IND-CCA-C$ secure, we have $Adv_{Const. 5, \mathcal{A}}^{simDV+S-OTR}(\kappa) \leq \text{negl}(\kappa)$. Therefore, Construction 5 is simDV+S-OTR secure. $\square$

Theorem 29. *If PrivMDVS is weak-Unf secure, Construction 5 is strong-Unf secure.*

Proof. Let $\mathcal{A}$ be a PPT adversary that breaks the strong-Unf security of Construction 5. We show a PPT adversary $\mathcal{B}$ that breaks the weak-Unf security of PrivMDVS by using $\mathcal{A}$, as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}$ generates $pp_{PKE} \leftarrow PKE.Setup(1^\kappa)$ and executes $\mathcal{A}$ on input (pp_{MDVS}, pp_{PKE}) .
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ as follows.
 - $O_{SPK}()$, $O_{SSK}(spk)$: $\mathcal{B}$ answers oracle queries by using its $O_{SPK}()$ and $O_{SSK}(spk)$ oracles, respectively.
 - $O_{VPK}()$: $\mathcal{B}$ generates $\hat{vpk}$ via its $O_{SPK}()$ oracle and $(pk, sk) \leftarrow PKE.KeyGen(pp_{PKE})$. Then, $\mathcal{B}$ returns $vpk := (\hat{vpk}, pk)$ to $\mathcal{A}$.
 - $O_{VSK}(vpk)$: $\mathcal{B}$ gets $\hat{vsk}$ corresponding to $\hat{vpk}$ via its $O_{VSK}()$ oracle, and returns $vsk := (\hat{vsk}, sk)$ to $\mathcal{A}$.
 - $O_{Sig}(spk, \mathcal{D}, m)$: $\mathcal{B}$ generates a signature using $PrivMDVS.Sign$ except that each $\hat{\sigma}_i$ is generated by querying $(spk, \{\hat{vpk}_i\}, m || \mathcal{D})$ to its O_{Sig} oracle.
 - $O_{Ver}(spk, vpk, \mathcal{D}, m, \sigma)$: $\mathcal{B}$ verifies the signature as the $PrivVrfy$ algorithm except that $\hat{\sigma}$ is verified by querying $(spk, \hat{vpk}, \{\hat{vpk}\}, m || \mathcal{D}, \hat{\sigma})$ to its O_{Ver} oracle.

3. Finally, $\mathcal{A}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^*)$ as forgery. $\mathcal{B}$ checks if $\mathcal{A}$ breaks the **strong-Unf** security of Construction 5. If this is the case, then there exists $\text{vpk}_i \in \mathcal{D}^*$ such that $\hat{\text{vpk}}_i \notin L_{\text{VSK}}$ and

$$\text{PrivMDVS.PrivVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}^*, \text{vsk}_i, \{\hat{\text{vpk}}_i\}, \text{m}^* \parallel \mathcal{D}^*, \hat{\sigma}_i^*) = 1.$$

Then, $\mathcal{B}$ outputs $(\text{spk}^*, \{\hat{\text{vpk}}_i\}, \text{m}^* \parallel \mathcal{D}^*, \hat{\sigma}_i^*)$ as its forgery.

We can verify that $\mathcal{B}$ perfectly simulates the **strong-Unf** game against $\mathcal{A}$. Moreover, if $\mathcal{A}$'s output satisfies the condition in line 4 of the **strong-Unf** game, then the $\mathcal{B}$'s output also satisfies the condition in line 4 of the **weak-Unf** game. Therefore, if $\mathcal{A}$ breaks the **strong-Unf** security of Construction 5, $\mathcal{B}$ breaks the **weak-Unf** security of PrivMDVS, i.e.,

$$\text{Adv}_{\text{Const. 5}, \mathcal{A}}^{\text{strong-Unf}}(\kappa) = \text{Adv}_{\text{PrivMDVS}, \mathcal{B}}^{\text{weak-Unf}}(\kappa).$$

Since PrivMDVS is **weak-Unf** secure, we have $\text{Adv}_{\text{Const. 5}, \mathcal{A}}^{\text{strong-Unf}}(\kappa) \leq \text{negl}(\kappa)$, that is, Construction 5 is **strong-Unf** secure. $\square$

Remark 30 (On the tightness of security proof). The reduction incurs a non-constant polynomial loss $2|\mathcal{C}^*|$ for the **allDV+S-OTR** security of the underlying PrivMDVS. However, this is not the case if it has OTR property against *computationally unbounded* adversaries. For example, our PrivMDVS from RS (which is a combination of Construction 3 and 4) has this property if RS has anonymity against computationally unbounded adversaries. Therefore, combining such an RS scheme with a tight N -IND-CCA-C secure PKE scheme and a tight N -sEUf-CMA secure OTS scheme, we obtain a tightly-secure (**strong, simDV+S**)-PrivMDVS (without consistency). Such tightly-secure building blocks can be instantiated by tightly-secure RS schemes [LPQ18, HT21], tightly-secure PKE schemes [LLP20], and tightly-secure strong signature schemes [DGJL21].

9 Generic Conversion for Adding Consistency to PrivMDVS

We present a universal conversion for incorporating consistency into any **strong-Unf** secure or **allDV+S-OTR** secure PrivMDVS. Our approach builds upon the methodology introduced by Damgård et al. [DHM⁺20], who employed non-interactive zero-knowledge (NIZK) arguments to guarantee the correct generation of signatures by the signing or simulation algorithm within their particular PrivMDVS scheme. We generalize their technique to make it applicable to *any* **strong-Unf** secure or **allDV+S-OTR** secure PrivMDVS schemes. Note that, to make the converted signature non-malleable, the MDVS signature and NIZK proof are bound by a strong one-time signature, similar to Construction 5.

Construction 6. Let PrivMDVS be a PrivMDVS scheme and NIZK an NIZK argument for the language

$$\begin{aligned} \mathcal{L}_{\text{Con}} := & \{(\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \text{m}, \sigma) \mid \\ & \exists(\text{ssk}, r) \text{ s.t. } \sigma = \text{PrivMDVS.Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}, \mathcal{D}, \text{m}; r) \\ & \vee \exists(\mathcal{C}, r) \text{ s.t. } \sigma = \text{PrivMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m}; r)\}. \end{aligned}$$

The scheme is as follows.

Setup(1^κ): Compute $\text{pp}_{\text{MDVS}} \leftarrow \text{PrivMDVS.Setup}(1^\kappa)$ and $\text{crs} \leftarrow \text{NIZK.Setup}(1^\kappa)$. Output $\text{pp} := (\text{pp}_{\text{MDVS}}, \text{crs})$.

SKGen(pp): Output $(\text{spk}, \text{ssk}) \leftarrow \text{PrivMDVS.SKGen}(\text{pp}_{\text{MDVS}})$.

VKGen(pp): Output $(\text{vpk}, \text{vsk}) \leftarrow \text{PrivMDVS.VKGen}(\text{pp}_{\text{MDVS}})$.

Sign($\text{pp}, \text{ssk}, \mathcal{D}, \text{m}$): Firstly, generate $(\text{ovk}, \text{osk}) \leftarrow \text{OTS.KeyGen}(1^\kappa)$. Compute $\hat{\sigma} \leftarrow \text{PrivMDVS.Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}, \mathcal{D}, \text{m} \parallel \text{ovk}; r)$ with randomness r . Set $x := (\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \text{m} \parallel \text{ovk}, \hat{\sigma})$ and $w := (\text{ssk}, r)$ and compute $\pi \leftarrow \text{NIZK.Prove}(\text{crs}, x, w)$ and $\text{osig} \leftarrow \text{OTS.Sign}(\text{osk}, \hat{\sigma} \parallel \pi)$. Output $\sigma := (\hat{\sigma}, \pi, \text{ovk}, \text{osig})$.

$\text{PrivVrfy}(\text{pp}, \text{spk}, \text{vsk}, \mathcal{D}, \text{m}, \sigma) : \text{Parse}(\hat{\sigma}, \pi, \text{ovk}, \text{osig}) := \sigma$. Set $x := (\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \text{m} \parallel \text{ovk}, \hat{\sigma})$ and compute $b_1 \leftarrow \text{NIZK.Verify}(\text{crs}, x, \pi)$, $b_2 \leftarrow \text{PrivMDVS.PrivVrfy}(\text{pp}_{\text{MDVS}}, \text{spk}, \text{vsk}, \mathcal{D}, \text{m} \parallel \text{ovk}, \hat{\sigma})$ and $b_3 \leftarrow \text{OTS.Verify}(\text{ovk}, \hat{\sigma} \parallel \pi, \text{osig})$. If $b_1 = b_2 = b_3 = 1$ holds, then output 1; otherwise output 0.

$\text{Sim}(\text{pp}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m})$: Firstly, generate $(\text{ovk}, \text{osk}) \leftarrow \text{OTS.KeyGen}(1^\kappa)$. Compute $\hat{\sigma} \leftarrow \text{PrivMDVS.Sim}(\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \mathcal{C}, \text{m} \parallel \text{ovk}; r)$ with randomness r . Set $x := (\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \text{m} \parallel \text{ovk}, \hat{\sigma})$ and $w := (\mathcal{C}, r)$ and compute $\pi \leftarrow \text{NIZK.Prove}(\text{crs}, x, w)$ and $\text{osig} \leftarrow \text{OTS.Sign}(\text{osk}, \hat{\sigma} \parallel \pi)$. Output $\sigma := (\hat{\sigma}, \pi, \text{ovk}, \text{osig})$.

We first show Construction 6 has consistency.

Theorem 31. *If PrivMDVS is correct and strong-Unf secure or allDV+S-OTR secure, and NIZK is sound, Construction 6 has consistency.*

Proof. Let $\mathcal{A}$ be a PPT adversary that breaks the consistency of Construction 6. We show a PPT adversary $\mathcal{B}$ that breaks the soundness of NIZK by using $\mathcal{A}$, as follows.

1. Upon receiving crs , $\mathcal{B}$ generates $\text{pp}_{\text{MDVS}} \leftarrow \text{PrivMDVS.Setup}(1^\kappa)$ and executes $\mathcal{A}$ on input $(\text{pp}_{\text{MDVS}}, \text{crs})$.
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ following the protocol description.
3. Finally, $\mathcal{A}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^* = (\hat{\sigma}^*, \pi^*, \text{ovk}^*, \text{osig}^*))$ as its forgery. $\mathcal{B}$ checks if $\mathcal{A}$ breaks consistency (i.e., checks the condition in line 4). If this is the case, then $\mathcal{B}$ outputs $x^* := (\text{pp}_{\text{MDVS}}, \text{spk}^*, \mathcal{D}^*, \text{m}^* \parallel \text{ovk}^*, \hat{\sigma}^*)$ and π^* .

We can verify that $\mathcal{B}$ perfectly simulates the consistency game against $\mathcal{A}$. We show that $\mathcal{B}$ breaks the soundness of NIZK when $\mathcal{A}$ breaks the consistency of Construction 6. Since one of two uncorrupted verifiers vpk_0 and vpk_1 accepts the signature σ^* (i.e., the verification algorithm outputs 1), we have $\text{OTS.Verify}(\text{ovk}^*, \hat{\sigma}^* \parallel \pi^*, \text{osig}^*) = 1$ and $\text{NIZK.Verify}(\text{crs}, x^*, \pi^*) = 1$ by construction. Since they verify the same OTS signature and the same NIZK proof in the same manner, both uncorrupted verifiers have $b_1 = b_3 = 1$. In other words, the difference in the final decision between vpk_0 and vpk_1 is caused by the verification of $\hat{\sigma}^*$, i.e., the one accepts it, but the other rejects it. However, statements including such a $\hat{\sigma}^*$ satisfy $x^* \notin \mathcal{L}_{\text{Con}}$ since if $x^* \in \mathcal{L}_{\text{Con}}$, the validity of π^* ensures that uncorrupted verifiers obtain the same verification result from $\hat{\sigma}^*$ with overwhelming probability: If $\hat{\sigma}^*$ is generated by **Sign**, correctness ensures that both users outputs 1. If $\hat{\sigma}^*$ is generated by **Sim**, uncorrupted verifiers output 0 when PrivMDVS is strong-Unf secure (cf. Theorem 7) or 1 when PrivMDVS is allDV+S-OTR secure (cf. Theorem 10).¹¹ Therefore, if $\mathcal{A}$ breaks the consistency, $\mathcal{B}$ breaks the soundness of NIZK with overwhelming probability. That is, we have $\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{Con}}(\kappa) \leq \text{negl}(\kappa)$. $\square$

We then show that Construction 6 offers the other security properties. Especially, it inherits the unforgeability and OTR property of PrivMDVS if NIZK is zero-knowledge.

Theorem 32. *If PrivMDVS, OTS and NIZK are correct, Construction 6 is correct.*

Theorem 33. *Let $\text{xxx} \in \{\text{weak}, \text{strong}\}$. If PrivMDVS is xxx-Unf secure and NIZK is zero-knowledge, Construction 6 is xxx-Unf secure.*

Proof. Let $\mathcal{A}$ be a PPT adversary that breaks the xxx-Unf security of Construction 6. To prove the theorem, we consider the following sequence of games Game_0 and Game_1 .

Game_0 : This is the original xxx-Unf game.

Game_1 : In this game, CRS and NIZK proofs are generated as $(\overline{\text{crs}}, \bar{\tau}) \leftarrow \text{Sim}_1(1^\kappa)$ and $\pi \leftarrow \text{Sim}_2(\overline{\text{crs}}, \bar{\tau}, x)$.

By definition, we have $\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{xxx-Unf}}(\kappa) = \Pr[\text{Game}_0(\kappa) = 1] \leq |\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| + \Pr[\text{Game}_1(\kappa) = 1]$. We can upper-bound $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]|$ and $\text{Game}_1(\kappa) = 1$ by the following lemmas.

¹¹As shown in Theorem 12, strong-Unf security and allDV+S-OTR security are incompatible. Thus, PrivMDVS satisfies either strong-Unf security or allDV+S-OTR security.

Lemma 7. Game_0 and Game_1 are indistinguishable assuming NIZK is zero-knowledge. That is,

$$|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \text{negl}(\kappa).$$

Proof. We show a PPT adversary $\mathcal{B}$ that breaks the zero-knowledge property of NIZK by using $\mathcal{A}$, as follows.

1. Upon receiving crs , $\mathcal{B}$ generates $\text{pp}_{\text{MDVS}} \leftarrow \text{PrivMDVS.Setup}(1^\kappa)$ and executes $\mathcal{A}$ on input (pp, crs) . Note that crs is real or simulated CRS depending on the hidden challenge bit for $\mathcal{B}$.
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ by itself except that NIZK proofs are generated by sending (x, w) to its oracle.
3. Finally, $\mathcal{A}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^*)$ as forgery. If $\mathcal{A}$ breaks the xxx-Unf security, $\mathcal{B}$ outputs 1.

We can verify that $\mathcal{B}$ perfectly simulates the xxx-Unf game against $\mathcal{A}$. In addition, if the hidden challenge bit is 0, $\mathcal{A}$ is in Game_0 , else it is in Game_1 . Thus, due to the zero-knowledge property of NIZK, we have $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \text{negl}(\kappa) \leq \text{negl}(\kappa)$. $\square$

Lemma 8. There exists an adversary $\mathcal{B}$ such that $\Pr[\text{Game}_1(\kappa) = 1] = \text{Adv}_{\text{PrivMDVS}, \mathcal{B}}^{\text{xxx-Unf}}(\kappa)$.

Proof. We show an adversary $\mathcal{B}$ that breaks the xxx-Unf security of PrivMDVS by using $\mathcal{A}$, as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}$ generates $(\overline{\text{crs}}, \bar{\tau}) \leftarrow \text{Sim}_1(1^\kappa)$ and executes $\mathcal{A}$ on input $(\text{pp}_{\text{MDVS}}, \overline{\text{crs}})$.
2. $\mathcal{B}$ answers oracle queries from $\mathcal{A}$ by using its corresponding oracles. When generating signatures, $\mathcal{B}$ creates $\hat{\sigma}$ with its signing oracle and then creates an NIZK proof as $\pi \leftarrow \text{Sim}_2(\overline{\text{crs}}, \bar{\tau}, x)$. (Note that $\mathcal{B}$ knows $x = (\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \text{m} \parallel \text{ovk}, \hat{\sigma})$.)
3. Finally, $\mathcal{A}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \sigma^*)$ as forgery. $\mathcal{B}$ checks if $\mathcal{A}$ breaks the xxx-Unf security. If this is the case, then $\mathcal{B}$ outputs $(\text{spk}^*, \mathcal{D}^*, \text{m}^*, \hat{\sigma}^*)$ as its forgery.

We can verify that $\mathcal{B}$ perfectly simulates Game_1 against $\mathcal{A}$. Moreover, if the $\mathcal{A}$'s output satisfies the condition in line 4 of the xxx-Unf game, then the $\mathcal{B}$'s output also satisfies the condition in line 4 of the weak-Unf game. Therefore, if $\mathcal{A}$ breaks the xxx-Unf security in Game_1 , $\mathcal{B}$ breaks the same security of PrivMDVS, i.e., $\Pr[\text{Game}_1(\kappa) = 1] = \text{Adv}_{\text{PrivMDVS}, \mathcal{B}}^{\text{xxx-Unf}}(\kappa)$. $\square$

Putting everything together, we have

$$\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{xxx-Unf}}(\kappa) \leq \text{Adv}_{\text{PrivMDVS}, \mathcal{B}}^{\text{xxx-Unf}}(\kappa) + \text{negl}(\kappa).$$

Since PrivMDVS is xxx-Unf secure, we have $\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{xxx-Unf}}(\kappa) \leq \text{negl}(\kappa)$. Therefore, Construction 6 is xxx-Unf secure. $\square$

Theorem 34. Let $\text{yyy} \in \{\text{simDV}, \text{simDV}+\text{S}, \text{allDV}+\text{S}\}$. If PrivMDVS is yyy-OTR secure, OTS is sEUF-CMA secure and NIZK is zero-knowledge, Construction 6 is yyy-OTR secure.

Proof. Let $\mathcal{A}$ be a PPT adversary that breaks the yyy-OTR security of Construction 6. To prove the theorem, we consider the following sequence of games.

Game₀: This is the original yyy-OTR game.

Game₁: In this game, O_{Ver} outputs 0 if the event Forge occurs.

Forge: For the received signature $\sigma = (\hat{\sigma}, \pi, \text{ovk}, \text{osig})$, it satisfies $\text{ovk} = \text{ovk}^*$, $(\hat{\sigma}, \pi, \text{osig}) \neq (\hat{\sigma}^*, \pi^*, \text{osig}^*)$, and $\text{OTS.Verify}(\text{ovk}, \hat{\sigma} \parallel \pi, \text{osig}) = 1$.

Game₂: In this game, CRS and NIZK proofs are generated as $(\overline{\text{crs}}, \bar{\tau}) \leftarrow \text{Sim}_1(1^\kappa)$ and $\pi \leftarrow \text{Sim}_2(\overline{\text{crs}}, \bar{\tau}, x)$, respectively.

Game₃: In this game, σ_0^* is generated as $\sigma_0^* \leftarrow \text{Sign}(\text{pp}_{\text{MDVS}}, \text{ssk}^*, \mathcal{D}^*, \mathbf{m}^*)$ instead of $\sigma_0^* \leftarrow \text{Sim}(\text{pp}_{\text{MDVS}}, \text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \mathbf{m}^*)$.

By definition, we have $\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{yyy-OTR}}(\kappa) = |\Pr[\text{Game}_0(\kappa) = 1] - 1/2|$. Therefore, the advantage of $\mathcal{A}$ is

$$\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{yyy-OTR}}(\kappa) \leq \sum_{i=1}^3 |\Pr[\text{Game}_{i-1}(\kappa) = 1] - \Pr[\text{Game}_i(\kappa) = 1]| + \left| \Pr[\text{Game}_3(\kappa) = 1] - \frac{1}{2} \right|.$$

Here, we have $|\Pr[\text{Game}_3(\kappa) = 1] - 1/2| = 0$ since both σ_0^* and σ_1^* have the same distribution in **Game₃**. It remains to show how each $|\Pr[\text{Game}_i(\kappa) = 1] - \Pr[\text{Game}_{i+1}(\kappa) = 1]|$ is upper-bounded. Concretely, we prove the following lemmas.

Lemma 9. *Game₀ and Game₁ are indistinguishable assuming OTS is sEUF-CMA secure. That is,*

$$|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \text{Adv}_{\text{OTS}, \mathcal{B}_1}^{\text{sEUF-CMA}}(\kappa).$$

Proof. Unless **Forge** occurs, **Game₀** and **Game₁** are identical. Thus, we have $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \Pr[\text{Forge}]$. We show that the probability of occurring **Forge** is negligible if OTS is sEUF-CMA secure.

We show an adversary $\mathcal{B}_1$ that breaks the sEUF-CMA security of OTS by using $\mathcal{A}$ that causes **Forge**. The description of $\mathcal{B}_1$ is as follows.

1. Upon receiving the challenge verification key ovk^* , $\mathcal{B}_1$ setups the public parameter pp of Construction 6 and executes $\mathcal{A}$ on input pp .
2. $\mathcal{B}_1$ answers oracle queries from $\mathcal{A}$ as in the previous game except that O_{Ver} answers as follows.
 - $\text{O}_{\text{Ver}}(\text{spk}, \text{vpk}, \mathcal{D}, \mathbf{m})$: $\mathcal{B}_1$ checks if **Forge** occurs for the received $\sigma = (\hat{\sigma}, \pi, \text{ovk}, \text{osig})$. If so, $\mathcal{B}$ outputs $(\text{ovk}, \hat{\sigma} \parallel \pi, \text{osig})$ as a forged signature. Otherwise, $\mathcal{B}$ proceeds identically to the previous game.
3. Upon receiving $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \mathbf{m}^*)$, $\mathcal{B}_1$ creates the challenge signature with the challenge OTS's verification key ovk^* . That is, $\mathcal{B}_1$ sends $(\text{ovk}^*, \hat{\sigma}, \pi)$ and receives osig . Other procedures are identical to the previous game.
4. Finally, $\mathcal{A}$ outputs a bit b' .

We can verify that $\mathcal{B}_1$ perfectly simulates **Game₁**. We assume **Forge** has occurred during the game. Then, $\mathcal{B}_1$ breaks the sEUF-CMA security of OTS when **Forge** occurs. Therefore, we have $|\Pr[\text{Game}_0(\kappa) = 1] - \Pr[\text{Game}_1(\kappa) = 1]| \leq \Pr[\text{Forge}] \leq \text{Adv}_{\text{OTS}, \mathcal{B}_1}^{\text{sEUF-CMA}}(\kappa)$. $\square$

Lemma 10. *Game₁ and Game₂ are indistinguishable assuming NIZK is zero-knowledge. That is,*

$$|\Pr[\text{Game}_1(\kappa) = 1] - \Pr[\text{Game}_2(\kappa) = 1]| \leq \text{negl}(\kappa).$$

Proof. We show a PPT adversary $\mathcal{B}_2$ that breaks the zero-knowledge property of NIZK by using $\mathcal{A}$, as follows.

1. Upon receiving crs , $\mathcal{B}_2$ generates $\text{pp}_{\text{MDVS}} \leftarrow \text{PrivMDVS.Setup}(1^\kappa)$ and executes $\mathcal{A}$ on input (pp, crs) . Note that crs is real or simulated CRS depending on the hidden challenge bit for $\mathcal{B}_2$.
2. $\mathcal{B}_2$ answers oracle queries from $\mathcal{A}$ by itself except that NIZK proofs are generated by sending (x, w) to its oracle.
3. When $\mathcal{A}$ requests the challenge signature, $\mathcal{B}_2$ creates the challenge except that the NIZK proof is generated by sending (x, w) to its oracle.
4. Finally, $\mathcal{A}$ outputs b' . If $b = b'$, $\mathcal{B}_2$ outputs 1.

We can verify that $\mathcal{B}_2$ perfectly simulates the yyy-OTR game against $\mathcal{A}$. In addition, if the hidden challenge bit is 0, $\mathcal{A}$ is in Game_1 , else it is in Game_2 . Thus, due to the zero-knowledge property of NIZK, we have $|\Pr[\text{Game}_1(\kappa) = 1] - \Pr[\text{Game}_2(\kappa) = 1]| \leq \text{negl}(\kappa)$. $\square$

Lemma 11. *Game₂ and Game₃ are indistinguishable assuming PrivMDVS is yyy-OTR secure. That is, $|\Pr[\text{Game}_2(\kappa) = 1] - \Pr[\text{Game}_3(\kappa) = 1]| = 2\text{Adv}_{\text{PrivMDVS}, \mathcal{B}_3}^{\text{yyy-OTR}}(\kappa)$.*

Proof. We show a PPT adversary $\mathcal{B}_3$ that breaks the yyy-OTR security of PrivMDVS by using $\mathcal{A}$, as follows.

1. Upon receiving pp_{MDVS} , $\mathcal{B}_3$ generates $(\overline{\text{crs}}, \bar{\tau}) \leftarrow \text{Sim}_1(1^\kappa)$ and executes $\mathcal{A}$ on input $(\text{pp}_{\text{MDVS}}, \overline{\text{crs}})$.
2. $\mathcal{B}_3$ answers oracle queries from $\mathcal{A}$ by using its corresponding oracles. Note that
 - when generating signatures, $\mathcal{B}_3$ creates $\hat{\sigma}$ with its signing oracle and then creates an NIZK proof as $\pi \leftarrow \text{Sim}_2(\overline{\text{crs}}, \bar{\tau}, x)$. (Note that $\mathcal{B}_3$ knows $x = (\text{pp}_{\text{MDVS}}, \text{spk}, \mathcal{D}, \text{m}, \hat{\sigma})$.);
 - when $\text{O}_{\text{Ver}}(\text{vpk}, \text{vpk}, \mathcal{D}, \text{m}, \sigma)$ is queried one of the following cases occurs (since Froge does not occur):
 - (a) if $\text{ovk} = \text{ovk}^*$, $\hat{\sigma} = \hat{\sigma}^*$. In this case, to satisfy the winning condition in Line 13 (in simDV-OTR or simDV+S-OTR game), it must hold that $(\text{vpk}, \mathcal{D}, \text{m}) \neq (\text{vpk}^*, \mathcal{D}^*, \text{m}^*)$ or $\text{vpk} \in \mathcal{C}_{\text{vpk}}$. Thus, $\mathcal{B}_3$ can use its verification oracle to verify $\hat{\sigma}$.
 - (b) if $\text{ovk} \neq \text{ovk}^*$, $\text{m} \parallel \text{ovk} \neq \text{m}^* \parallel \text{ovk}^*$. So, $\mathcal{B}_3$ can send $(\text{spk}, \text{vpk}, \mathcal{D}, \text{m} \parallel \text{ovk}, \hat{\sigma})$ to its verification oracle even if $\hat{\sigma} = \hat{\sigma}^*$ and $\sigma \neq \sigma^*$.

Thus, when O_{Ver} is queried, $\mathcal{B}_3$ can use its verification oracle to answer queries.

3. Upon receiving $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \text{m}^*)$, $\mathcal{B}_3$ checks the condition in line 6 of yyy-OTR game. Then, $\mathcal{B}_3$ outputs $(\text{spk}^*, \mathcal{D}^*, \mathcal{C}^*, \text{m}^*)$ to its challenger and receives the challenge $\hat{\sigma}^*$. $\mathcal{B}_3$ then generates an NIZK proof $\pi^* \leftarrow \text{Sim}_2(\overline{\text{crs}}, \bar{\tau}, x^*)$ and returns $\sigma^* := (\hat{\sigma}^*, \pi^*)$ to $\mathcal{A}$.
4. Finally, $\mathcal{A}$ outputs a bit b' . $\mathcal{B}_3$ outputs b' to its challenger.

We can verify that $\mathcal{B}_3$ perfectly simulates Game_2 if the challenge bit is 0 or Game_3 if the bit is 1. Moreover, if the $\mathcal{A}$'s activities satisfy the conditions in line 6 and line 13 of the yyy-OTR game, $\mathcal{B}_3$ so does. Therefore, we have $|\Pr[\text{Game}_2(\kappa) = 1] - \Pr[\text{Game}_3(\kappa) = 1]| = 2\text{Adv}_{\text{PrivMDVS}, \mathcal{B}_3}^{\text{yyy-OTR}}(\kappa)$. $\square$

Putting everything together, we have

$$\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{yyy-OTR}}(\kappa) \leq \text{Adv}_{\text{OTS}, \mathcal{B}_1}^{\text{eUF-CMA}}(\kappa) + 2\text{Adv}_{\text{PrivMDVS}, \mathcal{B}_3}^{\text{yyy-OTR}}(\kappa) + \text{negl}(\kappa).$$

Since PrivMDVS is yyy-OTR secure, we have $\text{Adv}_{\text{Const. 6}, \mathcal{A}}^{\text{yyy-OTR}}(\kappa) \leq \text{negl}(\kappa)$. Thus, Construction 6 is yyy-OTR secure. $\square$

10 Evaluating Our MDVS schemes

In this section, we analyze the efficiency of concrete instantiations of our PrivMDVS schemes. We constructed the following three PrivMDVS schemes.

Ours 1: (weak, allDV+S)-PrivMDVS with consistency from RS (Construction 3+Construction 4).

Ours 2: (strong, simDV+S)-PrivMDVS without consistency from RS, PKE, and OTS (Ours 1 + Construction 5).

Ours 3: (strong, simDV+S)-PrivMDVS with consistency from RS, PKE, OTS, and NIZK (Ours 2 + Construction 6).

We give two instantiations in both classical and post-quantum settings with 128-bit security, and estimate their signature size and computational costs on the signing algorithm Sign and the verification algorithm Vrfy . We use the schemes in Table 3 as the underlying building blocks to estimate efficiency. Details are as follows.

Table 3: Building blocks for efficiency evaluation.

Primitive	Classic	Post-quantum
RS	DualRing [YEL ⁺ 21]	
PKE	DHIES [ABR01]	Kyber [SAB ⁺ 22]
OTS	Schnorr [Sch90]	Dilithium [LDK ⁺ 22]
NIZK	Groth’s zk-SNARK [Gro16]	Aurora [BCR ⁺ 19]

Table 4: Efficiency evaluation of our schemes.

(a) Signature size.

Scheme	#Designated verifiers ($= \mu$)				PQ?
	2^1	2^3	2^6	2^{10}	
Ours 1	195 B	327 B	525 B	789 B	$\times$
	4.5 KB	4.6 KB	6.0 KB	31.2 KB	$\checkmark$
Ours 2	614 B	2168 B	16672 B	265312 B	$\times$
	17.9 KB	59.3 KB	445.7 KB	7069.7 KB	$\checkmark$
Ours 3	838 B	2392 B	16896 B	265536 B	$\times$
	191 KB	232.4 KB	618.8 KB	7242.8 KB	$\checkmark$

(b) Estimated runtime of Sign.

Scheme	#Designated verifiers ($= \mu$)				PQ?
	2^1	2^3	2^6	2^{10}	
Ours 1	8 ms	36 ms	266 ms	4118 ms	$\times$
	2348 ms	3015 ms	7247 ms	72920 ms	$\checkmark$
Ours 2	17 ms	67 ms	538 ms	8602 ms	$\times$
	4696 ms	18784 ms	150273 ms	2404362 ms	$\checkmark$
Ours 3	79017 ms	79068 ms	79538 ms	87602 ms	$\times$
	308696 ms	322784 ms	454273 ms	2708363 ms	$\checkmark$

(c) Estimated runtime of Vrfy.

Scheme	#Designated verifiers ($= \mu$)				PQ?
	2^1	2^3	2^6	2^{10}	
Ours 1	7 ms	16 ms	72 ms	999 ms	$\times$
	391 ms	507 ms	1473 ms	18222 ms	$\checkmark$
Ours 2	8 ms	8 ms	8 ms	8 ms	$\times$
	391 ms	391 ms	391 ms	391 ms	$\checkmark$
Ours 3	11 ms	11 ms	11 ms	11 ms	$\times$
	6691 ms	6691 ms	6691 ms	6691 ms	$\checkmark$

Ours 1 (weak, allDV+S)-PrivMDVS. Regarding Ours 1, all signing/verification keys and signatures are the same as ones of the underlying PrivMDVS scheme (that is, the underlying ring signature scheme due to the result in Section 6.1). Moreover, the signing/verification algorithms **Sign** and **Vrfy** are also the same as ones of the underlying PrivMDVS scheme. Thus, we get the signature size shown in Table 4a, the computational cost of **Sign** shown in Table 4b, and the computational cost of **Vrfy** shown in Table 4c, which are identical to the signature size, the computational cost of the signing algorithm, the computational cost of the verification algorithm of DualRing, respectively.¹²

Ours 2 (strong, simDV+S)-PrivMDVS without consistency. The signature of our second scheme (Ours 2) consists of μ PKE ciphertexts, one OTS verification key, and one OTS signature. Also, the PKE scheme encrypts a signature of the PrivMDVS scheme with one verifier and an OTS verification key. Moreover, regarding the computational costs on **Sign**, we need one invocation of the OTS key generation, μ times invocations of the PrivMDVS signature generation (in our case, the RS signature generation over the ring size 2) and the PKE encryption, and one invocation of OTS signing. From them, we can estimate the signature size and the computational costs on **Sign** and **Vrfy** in the classical and the post-quantum settings as follows.

Classical setting: In the classical setting, we can estimate the signature size and the computational costs on **Sign** and **Vrfy** by using the DL-based DualRing [YEL⁺21], the DHIES [ABR01], and the Schnorr signature [Sch90] as follows.

- Signature size: The ciphertext size of DHIES PKE is about $32 + N$ Byte when encrypting an N Byte message and the size of the Schnorr signature scheme’s verification key and signature is about 32 B and 64 B, respectively. Since the signature size of the PrivMDVS scheme based on DualRing RS is about 195 B, we need about 259 B for each ciphertext. Thus, the signature size in classical setting is $\mu \cdot 259 + 96$ B.
- Computational costs on **Sign**: The key generation time of the Schnorr signature is about 0.1 ms.¹³ Then, since the signature generation time of the DL-based DualRing over a ring size 2 is about 8.2 ms and the encryption time of the DHIES is about 0.2 ms, we need about $\mu \cdot 8.4$ ms when iterating them μ times. Then, the signature generation time of the Schnorr signature is about 0.1 ms. Thus, the computational cost of **Sign** in the classical setting is about $\mu \cdot 8.4 + 0.2$ ms.
- Computational costs on **Vrfy**: The verification of the Schnorr signature takes about 0.2 ms, the decryption of the DHIES takes about 0.1 ms, and the verification of the DL-based DualRing over a ring size 2 takes about 7.4 ms. Thus, the computational cost of **Vrfy** in the classical setting is about 7.7 ms.

Post-quantum setting: In the post-quantum setting, we can estimate the signature size and the computational costs on **Sign** and **Vrfy** by using the lattice-based DualRing [YEL⁺21], the Kyber [SAB⁺22], and the Dilithium [LDK⁺22] as follows.¹⁴

- Signature size: The ciphertext size of Kyber PKE is about $1.0 + N$ KB when encrypting an N KB message, and the size of Dilithium’s verification key and signature is about 1.4 KB and 2.7 KB, respectively. Since the signature size of the PrivMDVS scheme based on DualRing RS is about 4.5 KB, we need about 6.9 KB for each ciphertext. Thus, the signature size in the post-quantum setting is $\mu \cdot 6.9 + 4.1$ KB.

¹²Note that we estimate the computational costs of the signing/verification algorithm of DualRing by running the Python source code given in the original paper [YEL⁺21] on a machine with Apple(R) M2 processor and 24 GB RAM.

¹³The runtime for single exponentiation is estimated at about 0.1 ms on a machine with Apple(R) M2 processor and 24 GB RAM via the mcl library in <https://github.com/herumi/mcl>.

¹⁴We adopt the running time of Kyber and Dilithium provided by the open quantum-safe library in <https://openquantumsafe.org/benchmarking/>.

- Computational costs on **Sign**: The key generation cost of the Dilithium is about 0.06 ms. Then, since the signature generation cost of the lattice-based DualRing over a ring size 2 is about 2348 ms and the encryption cost of the Kyber is about 0.01 ms, we need about $\mu \cdot 2348.01$ ms when iterating them μ times. Then, the signature generation cost of the Dilithium is about 0.1 ms. Thus, the computational cost of **Sign** in the post-quantum setting is about $\mu \cdot 2348.01 + 0.16$ ms.
- Computational costs on **Vrfy**: The verification cost of the Dilithium is about 0.05 ms, the decryption cost of the Kyber is about 0.01 ms, and the verification cost of the lattice-based DualRing over a ring size 2^1 is about 390.6 ms. Thus, the computational cost of **Vrfy** in the post-quantum setting is about 390.66 ms.

Substituting the concrete value of μ , we obtain the signature sizes, the computational costs on **Sign**, and the computational costs on **Vrfy** shown in Table 4a, Table 4b, and Table 4c, respectively.

Ours 3 (strong, simDV+S)-PrivMDVS with consistency. The signature size of our third scheme (Ours 3) is the sum of the signature size of the second scheme and the size of the NIZK proof plus the size of OTS’s verification key and signature. When considering the classical setting, the size of Groth’s zk-SNARK scheme is about 128 B; when considering the post-quantum setting, the size of the Aurora is bounded by about 169 KB.¹⁵ As a result, we obtain the signature sizes shown in Table 4a.

Next, the communication costs of **Sign** and **Vrfy** of Ours 3 are provided in Tables 4b and 4c. Regarding the computational cost on **Sign** of Ours 3, the cost is the sum of the cost on **Sign** of Ours 2 and the proof generation cost of NIZK arguments plus the key generation and signature generation cost of OTS. Here, in the classical (resp., post-quantum) setting, the proof generation cost of Groth’s zk-SNARK (resp., the Aurora) is about 79000 ms (resp., 304000 ms). Regarding the computational cost on **Vrfy** of Ours 3, the cost is the sum of the cost on **Vrfy** of Ours 2 and the proof verification cost of NIZK arguments plus the signature verification cost of OTS. Here, in the classical (resp., post-quantum) setting, the proof generation cost of Groth’s zk-SNARK (resp., the Aurora) is about 3.4 ms (resp., 6300 ms).

11 Conclusion

In this paper, we explored subset simulatable MDVS and presented its thorough formalization framework. We proposed new constructions of such MDVS schemes based on ring signatures, offering a new construction paradigm. Leveraging the versatility of ring signatures, we can realize multiple MDVS schemes based on various assumptions. Additionally, we evaluated the communication costs of our schemes in both classical and post-quantum settings, demonstrating their practicality. Furthermore, we showed that MDVS satisfying the security definitions by Chakraborty et al. [CHMR23] implies IND-CCA secure public-key encryption. This result suggests that constructing such an MDVS scheme requires additional tools than those typically used for digital signatures. Several open problems remain. While this work focuses on the fundamental security properties of MDVS, other aspects, such as the privacy of signers and verifiers, also merit further exploration. Investigating these security properties is an important direction for future research. We consider publicly verifiable MDVS to be fully equivalent to ring signatures; however, it remains an open question whether an additional property could differentiate these two primitives. Additionally, all MDVS schemes with consistency, including ours, rely on NIZK, and thus, we leave whether consistency can be achieved without NIZK as an open problem. Finally, Yamashita and Hara [YH24] recently demonstrated the black-box separation of all simulatable privately verifiable MDVS from ring signatures. Whether a similar separation can be shown for subset simulatable privately verifiable MDVS remains an intriguing open question.

¹⁵We adopt concrete computation and communication costs of NIZKs provided in the previous work [ISW21]. Here, we consider a very rough estimation of the proof size and the running time of proof generation and verification by assuming a large circuit described by rank-one constraint systems (R1CS) with 2^{20} constraints.

Acknowledgments

The authors thank the anonymous reviewers of IEEE CSF 2024 and CSF 2025 for their constructive comments and suggestions and Yuki Fujita for pointing out errors in our security proofs and suggesting corrections. This research was partially supported by JST CREST JPMJCR22M1, Japan, JSPS KAKENHI Grant Numbers JP23H00468, JP23K16881, and JP24K20776, Japan, and “Research and development on new generation cryptography for secure wireless communication services” among “Research and Development for Expansion of Radio Wave Resources (JPJ000254),” which was supported by the Ministry of Internal Affairs and Communications, Japan.

References

- [ABR01] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. The oracle Diffie-Hellman assumptions and an analysis of DHIES. In David Naccache, editor, *CT-RSA 2001*, volume 2020 of *LNCS*, pages 143–158. Springer, Berlin, Heidelberg, April 2001.
- [BCR⁺19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 103–128. Springer, Cham, May 2019.
- [BFG⁺22] Jacqueline Brendel, Rune Fiedler, Felix Günther, Christian Janson, and Douglas Stebila. Post-quantum asynchronous deniable key exchange and the Signal handshake. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part II*, volume 13178 of *LNCS*, pages 3–34. Springer, Cham, March 2022.
- [BKM06] Adam Bender, Jonathan Katz, and Ruggero Morselli. Ring signatures: Stronger definitions, and constructions without random oracles. In Shai Halevi and Tal Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 60–79. Springer, Berlin, Heidelberg, March 2006.
- [CHMR23] Suviradip Chakraborty, Dennis Hofheinz, Ueli Maurer, and Guilherme Rito. Deniable authentication when signing keys leak. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 69–100. Springer, Cham, April 2023.
- [Cho08] Sherman S. M. Chow. Multi-designated verifiers signatures revisited. *International Journal of Network Security*, 7(3):348–357, November 2008.
- [DGJL21] Denis Diemert, Kai Gellert, Tibor Jager, and Lin Lyu. More efficient digital signatures with tight multi-user security. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 1–31. Springer, Cham, May 2021.
- [DHM⁺20] Ivan Damgård, Helene Haagh, Rebekah Mercer, Anca Nitulescu, Claudio Orlandi, and Sophia Yakoubov. Stronger security and constructions of multi-designated verifier signatures. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 229–260. Springer, Cham, November 2020.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.
- [HKKP22] Keitaro Hashimoto, Shuichi Katsumata, Kris Kwiatkowski, and Thomas Prest. An efficient and generic construction for Signal’s handshake (X3DH): Post-quantum, state leakage secure, and deniable. *Journal of Cryptology*, 35(3):17, July 2022.

- [HT21] Keisuke Hara and Keisuke Tanaka. Tightly secure ring signatures in the standard model. *Theoretical Computer Science*, 892:208–237, 2021.
- [HZM⁺24] Zhengan Huang, Gongxian Zeng, Xin Mu, Yu Wang, and Yue Yu. Multi-designated detector watermarking for language models, 2024.
- [ISW21] Yuval Ishai, Hang Su, and David J. Wu. Shorter and faster post-quantum designated-verifier zkSNARKs from lattices. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021*, pages 212–234. ACM Press, November 2021.
- [LDK⁺22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [LLP20] Youngkyung Lee, Dong Hoon Lee, and Jong Hwan Park. Tightly CCA-secure encryption scheme in a multi-user setting with corruptions. *Designs, Codes and Cryptography*, 88(11):2433–2452, November 2020.
- [LPQ18] Benoît Libert, Thomas Peters, and Chen Qian. Logarithmic-size ring signatures with tight security from the DDH assumption. In Javier López, Jianying Zhou, and Miguel Soriano, editors, *ESORICS 2018, Part II*, volume 11099 of *LNCS*, pages 288–308. Springer, Cham, September 2018.
- [LSMP07] Yong Li, Willy Susilo, Yi Mu, and Dingyi Pei. Designated verifier signature: Definition, framework and new constructions. In Jadwiga Indulska, Jianhua Ma, Laurence T. Yang, Theo Ungerer, and Jiannong Cao, editors, *Ubiquitous Intelligence and Computing*, pages 1191–1200, 2007.
- [LV04] Fabien Laguillaumie and Damien Vergnaud. Multi-designated verifiers signatures. In Javier López, Sihan Qing, and Eiji Okamoto, editors, *ICICS 04*, volume 3269 of *LNCS*, pages 495–507. Springer, Berlin, Heidelberg, October 2004.
- [LV07] Fabien Laguillaumie and Damien Vergnaud. Multi-designated verifiers signatures: anonymity without encryption. *Information Processing Letters*, 102(2):127–132, 2007.
- [MPR21] Ueli Maurer, Christopher Portmann, and Guilherme Rito. Giving an adversary guarantees (or: How to model designated verifier signatures in a composable framework). In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 189–219. Springer, Cham, December 2021.
- [MPR22] Ueli Maurer, Christopher Portmann, and Guilherme Rito. Multi-designated receiver signed public key encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 644–673. Springer, Cham, May / June 2022.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [Sch90] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 239–252. Springer, New York, August 1990.
- [SKS06] G. Shailaja, K. Phani Kumar, and Ashutosh Saxena. Universal designated multi verifier signature without random oracles. In *9th International Conference on Information Technology (ICIT’06)*, pages 168–171, 2006.

- [TGL⁺19] Nirvan Tyagi, Paul Grubbs, Julia Len, Ian Miers, and Thomas Ristenpart. Asymmetric message franking: Content moderation for metadata-private end-to-end encryption. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 222–250. Springer, Cham, August 2019.
- [YEL⁺21] Tsz Hon Yuen, Muhammed F. Esgin, Joseph K. Liu, Man Ho Au, and Zhimin Ding. DualRing: Generic construction of ring signatures with efficient instantiations. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 251–281, Virtual Event, August 2021. Springer, Cham.
- [YH24] Kyosuke Yamashita and Keisuke Hara. On the black-box impossibility of multi-designated verifiers signature schemes from ring signature schemes. *Journal of Mathematical Cryptology*, 18(1), 2024.
- [ZAYS12] Yunmei Zhang, Man Ho Au, Guomin Yang, and Willy Susilo. (strong) multi-designated verifiers signatures secure against rogue key attack. In Li Xu, Elisa Bertino, and Yi Mu, editors, *Network and System Security*, pages 334–347, 2012.